

Vue 基础面试题速记通关版 _ 面试刷题 mianshiya.com

本资源来自面试鸭: <https://www.mianshiya.com>

推荐更多免费学编程资源:

1. 编程导航学习网站: [学编程、做项目、拿 Offer!](#)
2. 企业高频面试题库: [开始刷题, 面试遇原题!](#)
3. 精选简历模板大全: [1 分钟搞定简历!](#)
4. AI 资源导航网站: [获取最新 AI 黑科技!](#)
5. 1 对 1 模拟面试: [随时随地提升面试能力](#)

Vue Router 的 hash 模式和 history 模式有什么区别?

Vue Router 的路由模式选择, 本质上是前端路由如何映射 URL 和页面状态的问题。两种模式在用户体验和实现机制上有明显差异。

hash 模式依赖 URL 中的 # 后面的部分, 比如 `example.com/#/home`。这部分变化不会触发浏览器向服务器发起请求, 所以前端可以自由控制路由跳转, 后端压根不经过。兼容性好, IE8 也能用, 适合老项目或对部署要求低的场景。

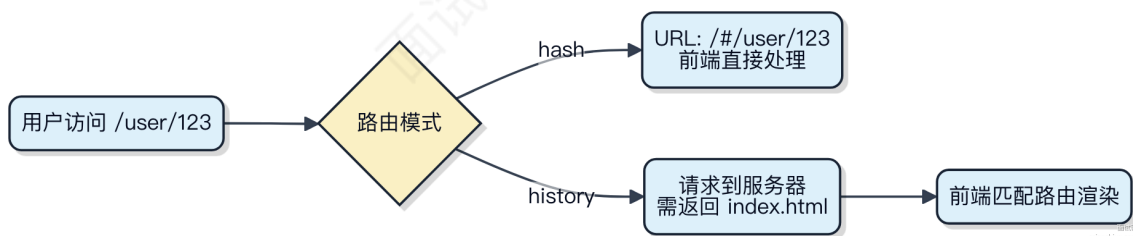
history 模式更优雅, URL 看起来就是标准路径, 像 `example.com/home`。但它要求后端配合, 所有路由都得指向同一个入口文件 (如 `index.html`), 否则用户直接访问这个路径会 404。Nginx 配置要加 `try_files $uri $uri/ /index.html;`, Node.js 服务也要处理通配路由。

代码上切换很简单:

```
const router = new VueRouter({
  mode: 'history', // 或 'hash'
  routes
})
```

- 1) hash 模式的 # 是锚点标识, 本来用于页面内定位
- 2) history 模式利用了 HTML5 的 **History API** (`pushState`、`replaceState`) 来修改 URL 而不刷新页面
- 3) 单页应用用 history 模式体验更好, 但部署成本略高

如果服务端搞不定 fallback 路由, 就别强上 history 模式, 不然静态资源可能都加载不了。



DOM 渲染在哪个 Vue 生命周期钩子中就已经完成?

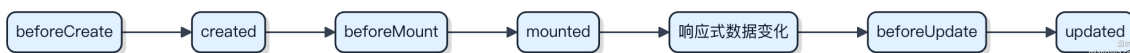
Vue 的生命周期里，模板编译之后生成的 VNode 会经过 patch 过程转换为真实 DOM。这个过程是在 `mounted` 钩子触发前完成的。

- 1) `mounted` 被调用时，组件已经挂载到页面上，对应的 DOM 节点此时已经存在，可以安全地进行 DOM 操作。比如你在这个钩子里访问 `this.$el`，拿到的就是真实的 DOM 元素。
- 2) 如果是有状态变化导致的后续更新，DOM 的重新渲染发生在 `updated` 钩子之前。但首次渲染完成的标志就是 `mounted`。
- 3) 需要注意的是，如果组件内有异步请求驱动的视图更新，或者使用了 `nextTick`，那么这些操作后的 DOM 更新需要在 `this.$nextTick()` 的回调里才能确保拿到最新 DOM 结构。

代码示例如下：

```
mounted() {  
  // 此时 DOM 已经渲染完毕  
  console.log(this.$el.innerHTML)  
},  
updated() {  
  // 响应式数据更新后，DOM 重新渲染完成  
}
```

对于包含子组件的情况，父组件的 `mounted` 并不保证所有子组件也已完成挂载。如果要等整个组件树都挂载完，可以用 `this.$nextTick` 结合 `app.mount()` 后的逻辑控制。



Vue 中 :class 和 :style 有几种表示方式？

:class 和 :style 本质是 Vue 对 DOM class 和 style 属性的动态绑定，核心在于支持灵活的数据驱动写法。

- 1) 对于 :class，三种主流写法。对象语法最直观，布尔值控制开关。

```
<div :class="{ active: isActive, 'text-danger': hasError }"></div>
```

数组语法适合类名动态切换。

```
<div :class="[activeClass, errorClass]"></div>
```

也可以混合用，对象和数组结合，应对复杂条件。

```
<div :class="[{ active: isActive }, errorClass]"></div>
```

- 2) :style 写法类似。对象语法直接内联样式，推荐驼峰式。

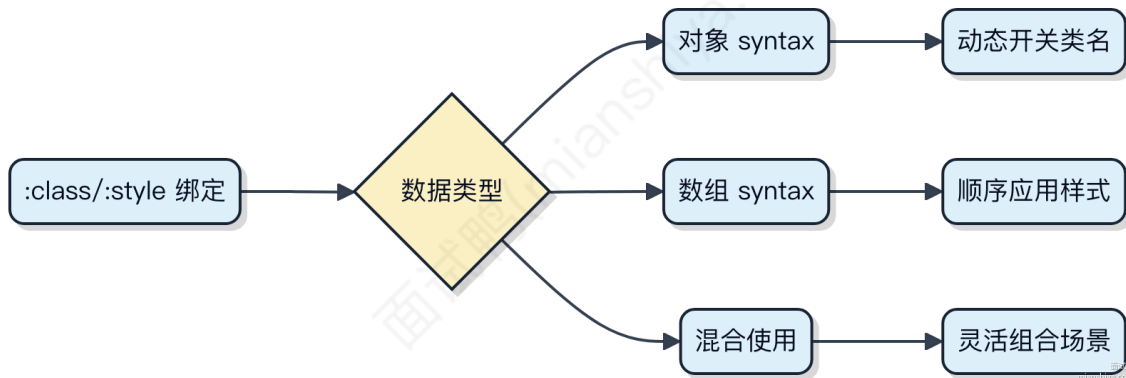
```
<div :style="{ color: activeColor, fontSize: fontSize + 'px' }"></div>
```

数组语法支持多组样式合并，常用于组件库封装。

```
<div :style="[baseStyles, overridingStyles]"></div>
```

Vue 会自动处理浏览器前缀，比如 transform 自动加 -webkit- 等。

注意点：频繁切换的样式建议写在 CSS 类中，用 :class 控制，而不是反复改 :style 对象，避免性能问题。内联样式更适合那些无法预知、运行时计算的值。



什么是 Vue Router 的 router-link 组件？

router-link 是 Vue Router 提供的一个声明式导航组件，用来替代普通的 a 标签跳转。你点它，页面不刷新，路由就变了，对应的视图也会更新。

它最终会渲染成一个 a 标签，但内部走的是 Vue Router 的导航逻辑，不会触发浏览器的整页 reload。比如你写 `<router-link to="/user">用户</router-link>`，点击后 URL 变成 `/user`，同时匹配并渲染对应的组件。

有几个常用属性值得注意：1) `to` 指定目标路径，支持字符串或对象形式，比如 `:to="{ name: 'User', params: { id: 8 } }"` 2) `tag` 可以指定渲染成什么标签（不过在 Vue 3 中已被移除）3) `active-class` 控制激活状态的 class 名，默认是 `router-link-active`

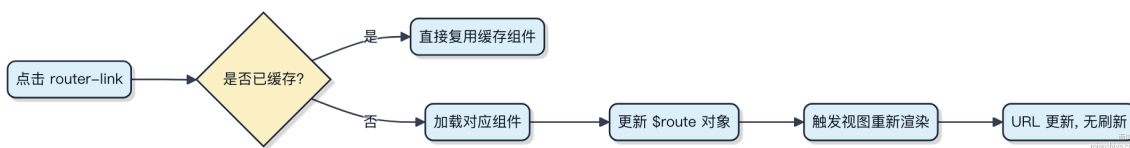
它内部自动帮你处理了当前路由匹配的高亮问题。比如你进了 `/user`，所有指向 `/user` 的 router-link 就会自动加上 `router-link-active`，嵌套路由还会带上 `router-link-exact-active`。

代码上其实很简单，核心就是声明式绑定：

```
<router-link to="/home">首页</router-link>
```

等价于用编程方式调用 `router.push('/home')`，但更简洁直观。

一般你在做单页应用的菜单、分页切换、面包屑导航时都会用到它。比起手动写 `click + this.$router.push`，它语义更强，也支持更多内置行为，比如通过 `replace` 属性触发 `replace` 导航，不会留下历史记录。



说说你对 Vue 的理解？相比于原生开发，使用 Vue 有哪些优点？

Vue 是一个用于构建用户界面的渐进式框架，它的设计思路是按需引入，你可以从一个小功能块开始用，也能撑起整个单页应用。数据绑定和组件化是它最基础的两块拼图。

Vue 通过响应式系统自动追踪依赖，在数据变化时精准更新视图，不用手动操作 DOM。你只需要关心状态怎么变，UI 就能同步跟上。比如用 `ref` 或 `reactive` 定义数据，模板里直接插值，改数据视图自动刷新。

1) 组件机制让 UI 能拆解成独立单元，每个组件封装了结构、样式和逻辑，复用和维护都方便。像 Element Plus 这类 UI 库就是基于 Vue 组件做的。2) 指令系统简化了 DOM 操作，`v-model` 实现表单双向绑定，`v-for` 渲染列表，比原生反复写 `addEventListener` 和 `innerHTML` 高效得多。3) 生态系统配套完整，Vue Router 管路由，Pinia 管全局状态，Vite 配合开发体验流畅，这些工具链原生开发得自己搭一套。

原生开发在项目变大后容易代码散落、依赖混乱，Vue 提供了一套约定大于配置的结构，团队协作更高效。当然小脚本或极致性能场景，直接操作 DOM 可能更轻量，但大多数业务项目，Vue 能帮你更快交付，也更容易维护。



对比 React 的函数式 + Hooks，Vue 的 Options API 更直观，Composition API 又提供了逻辑复用能力，学习曲线相对平缓。

Vue 中 computed 和 methods 的区别是什么？

计算属性和方法看着都能返回值，但底层设计目标完全不同。computed 是为了解决模板中逻辑复杂、重复计算的问题，它基于响应式依赖进行缓存。只要依赖的数据没变，多次访问 computed 属性会直接返回缓存结果，不会重新执行函数。

methods 就简单了，它就是个普通函数，每次调用必然执行，不带任何缓存机制。你在模板里调 5 次 methods，它就执行 5 次，哪怕参数和数据都没变。

举个例子，你有个数组要做 filter 和 map 处理，放在 methods 里，每次渲染都重新跑一遍；而 computed 只有在数组更新时才重新计算一次，性能优势明显。

1) computed 适用于依赖响应式数据的计算场景，比如拼接字符串、统计数量、过滤列表。2) methods 适合处理事件回调、发起网络请求这类不需要缓存、每次都要执行的操作。

```
export default {
  data() {
    return {
      firstName: 'John',
      lastName: 'Doe'
    }
  },
  computed: {
    fullName() {
      // 只有 firstName 或 lastName 改变时才重新执行
      return this.firstName + ' ' + this.lastName
    }
  },
  methods: {
```

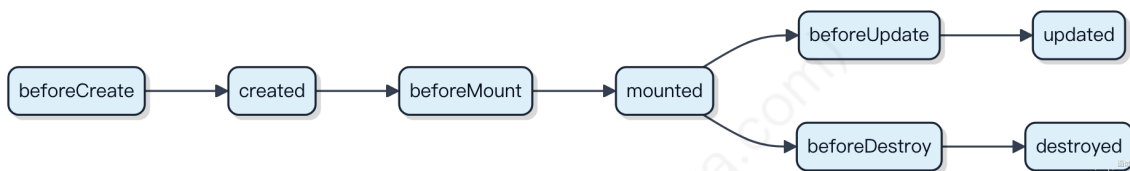
```
getFullName() {  
  // 每次调用都执行  
  return this.firstName + ' ' + this.lastName  
}  
}  
}
```

所以别把耗时计算扔 methods，尤其在模板里频繁使用时，很容易搞垮渲染性能。computed 才是做这种事的正确姿势。

什么是 Vue 的生命周期？生命周期的作用是什么？

Vue 的生命周期指的是一个组件从创建、更新到销毁的全过程，每个阶段都会触发对应的钩子函数。开发者可以在这些钩子中编写逻辑，控制组件的行为。

- 1) 创建阶段：beforeCreate 时数据观测还没开始，data 和方法都不可用；到了 created，数据已经挂载，可以访问 data、调用方法、发请求，但页面还没渲染，DOM 拿不到。
- 2) 挂载阶段：beforeMount 时模板编译完成，但还没把虚拟 DOM 渲染成真实 DOM；mounted 阶段真实节点已经插入页面，这时候可以操作 DOM，一般初始化第三方库比如 echarts 就放这。
- 3) 更新阶段：数据变化会触发 beforeUpdate 和 updated。这里要注意别在 updated 里改状态，容易死循环。
- 4) 销毁阶段：beforeDestroy 可以清理定时器、解绑事件、取消订阅；destroyed 之后实例完全被移除，所有绑定都解除了。



生命周期最大的作用就是让你知道“什么时候该干什么事”。比如 created 做数据初始化，mounted 做 DOM 操作，destroyed 做资源回收。搞不清顺序，就容易在错误的时机访问 DOM 或者内存泄漏。

通常在 Vue 的哪个生命周期钩子中请求异步数据？为什么？

一般在 created 钩子中请求异步数据，因为这个时候实例已经完成了数据观测、属性和方法的运算，但还没开始挂载 DOM。这时候发请求，能尽早拿到数据，页面一渲染就有内容，用户体验更好。

如果你等 mounted 再发请求，虽然也能工作，但会多等一轮渲染。尤其是首屏依赖接口数据时，延迟请求就意味着白屏时间更长。所以为了更快地拉取数据，created 是更合适的选择。

当然也有例外。比如你的接口依赖 DOM 元素（像需要获取某个节点的尺寸），那就得放到 mounted 里，因为 created 的时候 DOM 还没生成。

```
export default {  
  data() {  
    return {  
      list: []  
    }  
  }  
}
```

```
    }  
  },  
  created() {  
    // 实例一初始化完就发请求  
    this.fetchData()  
  },  
  methods: {  
    async fetchData() {  
      const res = await api.getList()  
      this.list = res.data  
    }  
  }  
}
```

- 1) `beforeCreate` 太早，数据和方法都还没准备好，不适合发请求
- 2) `created` 数据有了，DOM 没挂载，适合做数据预加载
- 3) `mounted` DOM 渲染完了，适合操作 DOM 或依赖视图的逻辑

如果组件支持服务端渲染（SSR），还得配合 `asyncData` 或类似机制，保证数据在服务端就提前获取。

在 Vue 的 v-for 循环中，key 有什么作用？

Vue 在更新由 v-for 生成的元素列表时，会尽可能复用和调整现有的 DOM 元素，而不是从头创建。这个过程靠的就是 key 来做节点的唯一标识。

- 1) 没有 key 的时候，Vue 会采用“就地更新”策略，简单按索引位置对比新旧节点，一旦列表顺序变了，比如插入、删除或排序，DOM 就可能复用错乱，导致状态异常，比如输入框内容错位。
- 2) 有了 key，Vue 就能精准识别每个节点的身份。比如用 id 作为 key，即便顺序变了，Vue 也能知道哪个元素是新增的，哪个是移除的，哪个只是移动了位置，从而正确触发组件的销毁、挂载或移动操作。
- 3) key 最好别用数组索引。虽然能避免警告，但一旦列表发生逆序、插入等操作，索引对应的 item 数据变了，Vue 会误以为那个 DOM 还能复用，结果就是组件状态不重置、动画错乱、输入状态残留等问题。

代码上长这样：

```
<li v-for="item in items" :key="item.id">  
  {{ item.name }}  
</li>
```

如果 item.id 是稳定唯一的，那 Vue 就能高效地追踪变化。换成 index，看着省事，实际埋坑。



Vue Router 支持哪几种路由模式？它们有什么区别？

Vue Router 主要支持两种路由模式：hash 模式和 history 模式，它们的核心差异在于 URL 形式和服务器依赖。

1) hash 模式利用 URL 中的 # 后面的部分作为路由路径，比如 /#/user。浏览器不会把 # 后的内容发送给服务器，所以无论怎么切换路由，请求压根不经过服务端，前端自己就能完全控制。这种模式兼容性好，IE9 都能用，开发时也不用担心 404。

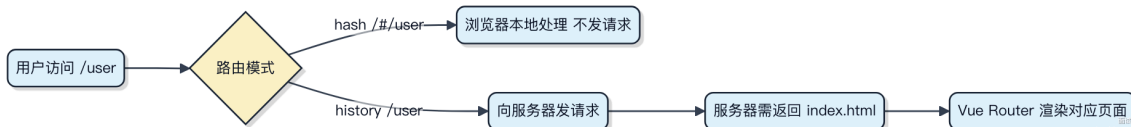
2) history 模式用的是原生的浏览器 History API，URL 看起来干净，像 /user 这样，没有 #。但问题来了，用户刷新页面或直接访问这个地址时，浏览器会向服务器发起请求。如果后端没做配置，返回 404 就搞不定了。所以用这种模式，后端必须做“兜底”，所有路由都指向 index.html。

代码上切换很简单：

```
const router = new VueRouter({
  mode: 'history', // 或 'hash'
  routes
})
```

一般新项目推荐 history 模式，用户体验更好，前提是能控制服务器配置。老项目或不想改后端的，直接上 hash 模式最省事。

要不要图示？来看看两种模式的请求流程差异：



v-on 在 Vue 中可以绑定多个方法吗？

v-on 当然能绑定多个方法，这在实际开发里太常见了。你可以在一个事件上挂多个处理函数，Vue 会按顺序执行。

比如点击按钮既要记录埋点，又要打开弹窗，直接写就行：

```
<button @click="trackClick, openModal">提交</button>
```

这两个方法都会被调用，`trackClick` 和 `openModal` 就是普通的 `methods` 里的函数。注意这里用的是逗号分隔，不是模板字符串里的 `${}` 那种语法。

如果逻辑复杂了，比如要传参数或者加条件判断，建议还是封装成一个新方法，在 `template` 里保持简洁。像这种带参数的：

```
<button @click="handleClick($event), track('submit')">提交</button>
```

虽然可行，但读起来就有点绕。尤其是多人协作时，别人一看得琢磨一下是不是有副作用。

还有一种情况是同一个元素监听不同事件，那更没问题了，`@click`、`@mouseover` 各自独立：

```
<div @click="onClick" @mouseover="onHover" @mouseleave="onLeave">hover me</div>
```

每个事件绑定自己的回调，互不影响。这也是 `Vue` 模板最基础的能力之一。

- 1) 同一个事件可以绑定多个方法，用逗号分隔
- 2) 不同事件各自绑定，天然支持
- 3) 复杂逻辑建议封装，别让模板太臃肿

其实本质就是 `DOM` 事件机制的封装，`Vue` 没做限制，怎么写顺手怎么来，但要顾及可维护性。

Vue 中 `computed` 和 `watch` 的区别是什么？

`computed` 和 `watch` 都是 `Vue` 中响应式数据变化的手段，但它们的定位和使用方式完全不同。

`computed` 是基于依赖缓存的**计算属性**。只要它依赖的数据没变，多次访问就直接返回缓存结果，不会重复执行函数。适用于模板中需要动态计算的值，比如 `fullName` 由 `firstName` 和 `lastName` 拼接而成。

```
computed: {
  fullName() {
    return this.firstName + ' ' + this.lastName
  }
}
```

`watch` 则是监听某个数据的变化，执行**副作用操作**。适合处理异步或开销较大的任务，比如监听搜索关键词，延迟 500ms 后发起请求。它不一定要返回值，更关注“变化后做什么”。

```
watch: {
  searchQuery(newVal) {
    this.debounceFetch(newVal)
  }
}
```

- 1) `computed` 有缓存，依赖不变时不会重新计算
- 2) `watch` 可以执行异步操作，`computed` 不行

3) watch 能监听深层次对象变化（配合 deep 选项），而 computed 只会追踪直接依赖

一般模板里用 computed，做异步或深度监听用 watch。别把本该 computed 做的事扔给 watch，那样代码会变得零碎难维护。

为什么 Vue 中给对象添加新属性后界面不刷新？

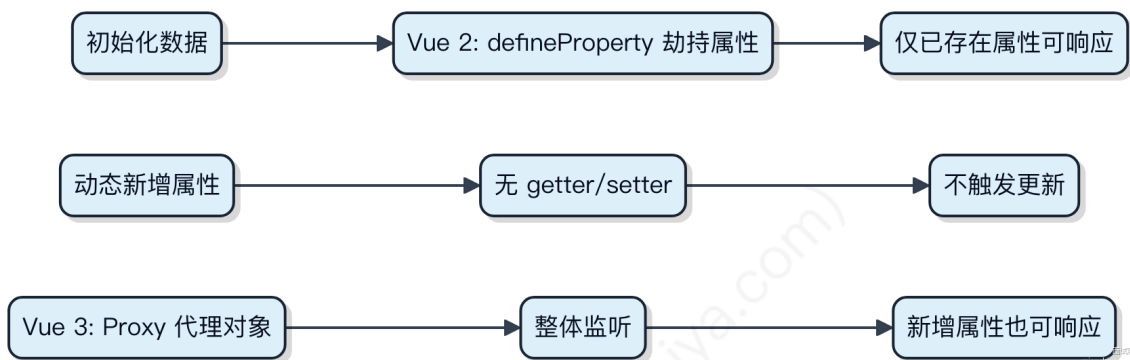
Vue 2 的响应式系统基于 `Object.defineProperty`，它在初始化时就劫持了对象所有属性的 getter 和 setter。后来添加的新属性，压根没有被定义过 get/set，自然不会触发依赖收集和派发更新。

你给 data 里的对象动态加个字段，比如 `this.obj.newProp = 'hi'`，这个 newProp 不会进入响应式流程。视图里用到 obj 的地方，根本不知道它变了。

解决办法有两个方向。一是提前声明好属性，哪怕初始值是 undefined。二是用 Vue 提供的 API 让新增的属性也能被追踪。比如用 `Vue.set(obj, 'newProp', 'hi')` 或实例方法 `this.$set(obj, 'newProp', 'hi')`，它们会确保新属性走一遍 defineReactive 流程。

如果是数组，直接通过下标赋值也不会触发更新，比如 `this.list[0] = newVal`。同样得用 `Vue.set` 或 `splice` 来处理。

Vue 3 改用 **Proxy** 后就没这个问题了。Proxy 能监听整个对象的访问和修改行为，不管是已有属性还是新增的，都能拦截到。所以 Vue 3 中给对象加属性，界面能正常刷新。



在 Vue 中，如果变量名以 _ 或 \$ 开头，会有什么问题？如何访问到这些值？

Vue 会对 data 中的属性做响应式处理，但会跳过一些特殊命名的字段。以 `_` 或 `$` 开头的变量名不会被代理到 Vue 实例上，你也拿不到它的 getter/setter。

- 1) `_` 开头的属性会被 Vue 忽略监听，主要是为了避免和 Vue 内部的私有属性、方法冲突。比如 Vue 自己用的 `_data`、`_events` 就是这种命名规则。
- 2) `$` 开头是给 Vue 实例方法和全局配置用的，像 `$emit`、`$nextTick`、`$refs` 都是官方保留的，自定义变量别占这个坑。

虽然这些字段不进 `this` 的代理层，但它们仍然存在于实例中，只是访问方式不一样。想拿到值，得通过 `this.$data` 原始对象去取。

```
export default {
  data() {
    return {
```

```

    _private: '私有变量',
    $internal: '内部标记'
  },
  mounted() {
    // 这样访问不到
    console.log(this._private)    // undefined
    console.log(this.$internal)   // undefined

    // 正确方式
    console.log(this.$data._private) // '私有变量'
    console.log(this.$data.$internal) // '内部标记'
  }
}

```

说白了，Vue 搞这层代理是为了实现响应式和模板解析，但这俩前缀相当于被“屏蔽”出了代理映射。数据还在，就是不让你直接点出来。真要用，绕个路就行。

当 Vue 的属性名称与 methods 中的方法名称一样时，会发生什么问题？

属性名称和 methods 里的方法重名时，Vue 不会报错，但会警告，最终 **methods** 里的函数会被挂载到实例上，而 **data** 里的同名属性会被覆盖。

- 1) 你定义一个 data 属性叫 `sayHello`，同时在 methods 也定义一个 `sayHello()` 方法，最终 `this.sayHello` 指向的是那个函数。
- 2) 当你在模板里写 `{{ sayHello }}`，Vue 会尝试执行这个函数，但函数没有返回值的话，模板就显示 `undefined`，容易让人懵。
- 3) 如果你不小心把 data 里的对象写成和 method 同名，比如 `user: {}` 和 `user()` 方法冲突，那 `user.name` 这种访问就会出错，因为 `user` 实际是个函数，不是对象。

这种情况其实在开发时 Vue 会在控制台抛 warning：

```
"Method "xxx" has already been defined as a data property."
```

所以别搞这种命名冲突，虽然 Vue 能跑起来，但结果不可控，维护的人会疯。

代码示例：

```

new Vue({
  data() {
    return {
      log: 'hello' // 这个会被干掉
    }
  },
  methods: {
    log() {
      console.log('called')
    }
  }
})

```

`this.log` 是函数，`this.log()` 可以调用，但 `{{ log }}` 会执行函数并试图读返回值，没返回就是空。

第一次加载 Vue 页面时会触发哪些生命周期钩子？

页面从无到有，Vue 实例初始化会走一套固定的流程。首次加载一个 Vue 页面时，会按顺序触发几个关键的生命周期钩子。

`new Vue()` 之后，实例开始创建。最先执行的是 `beforeCreate`，这时候数据观测还没 setup 好，`$data`、`methods` 都不能用。

接着进入 `created`，核心的数据响应式系统已经建立完成。**this** 指向实例，`data`、`methods`、`computed` 都可访问，异步请求一般就放这，避免阻塞渲染。

模板编译或 `render` 函数执行前，触发 `beforeMount`。此时页面还没挂载，`$el` 不可用，仍然停留在虚拟 DOM 阶段。

当 `VNode` 转成真实 DOM 并插入到 `el` 所指定的元素中后，`mounted` 钩子被调用。这是最早可以操作 DOM 的时机，常见于需要直接操控节点的场景，比如集成第三方库（如 `echarts` 初始化）。

- 1) `beforeCreate`
- 2) `created`
- 3) `beforeMount`
- 4) `mounted`



服务端渲染时，`beforeMount` 和 `mounted` 在服务端也会触发，但实际 DOM 操作在客户端才生效，要注意兼容性。组件销毁和更新阶段的钩子，这次不走。

Vue 常用的修饰符有哪些？分别有哪些应用场景？

v-model 修饰符

`v-model` 默认在 `input` 事件触发时同步数据，但有些场景需要控制更新时机。

`.lazy` 把绑定从 `input` 改成 `change`，适合输入框失焦再提交的场景，比如搜索关键词防抖。

`.number` 自动把用户输入转成数字类型，避免字符串拼接出问题，表单校验时特别有用。

`.trim` 自动过滤首尾空格，处理登录账号、邮箱这类对空白敏感的字段很实用。

事件修饰符

`@click.prevent` 阻止默认行为，比如表单提交时阻止页面刷新。

`@click.stop` 阻止冒泡，下拉菜单点击内部元素不希望触发外层关闭逻辑就得靠它。

`@click.self` 只有点击事件来源是当前元素本身才触发，和 `stop` 比更精确。

`.once` 确保事件只触发一次，适合弹窗确认、抽奖按钮这种防重复操作的场景。

键盘修饰符

`@keyup.enter` 绑定回车键，登录框输完密码直接敲回车登录。

`@keyup.esc` 处理退出逻辑，比如弹窗按 `Esc` 关闭。

组合写法如 `@keyup.ctrl.c` 监听 `Ctrl+C`，做富文本编辑器快捷键很方便。

代码示例：

```
<input v-model.trim="username" />
<button @click.once="submit">提交</button>
<input @keyup.enter="search" />
```

表单修饰符

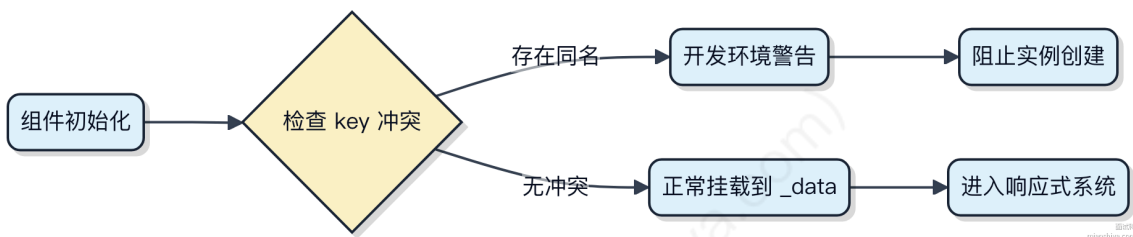
.native 用在自定义组件上，监听原生 DOM 事件，封装组件时透传事件监听。
.sync 是语法糖，实现父子组件 props 双向绑定，Vue 3 中被 v-model 参数替代。

Vue 计算属性的函数名和 data 中的属性可以同名吗？为什么？

不能同名，Vue 内部会把计算属性和 data 中的属性都挂载到同一个响应式对象上，同名会产生冲突。

- 1) 计算属性本质上是通过 `Object.defineProperty` 定义的 getter，它最终会和 data 里的属性一起被收集到组件的 `_data` 对象中。如果名字重复，后面的定义会覆盖前面的，导致行为异常或警告。
- 2) Vue 在初始化时会对 key 做校验，一旦发现 computed 和 data 存在同名属性，会在开发环境直接抛出警告，提示 "The computed property 'xxx' is already defined in data"，阻止这种不合理的定义。
- 3) 即便你强行绕过，比如用字符串拼接等方式隐藏命名冲突，也会导致响应式系统追踪错乱。比如模板中引用这个属性时，可能拿到的是 data 的初始值，而不是计算属性的动态结果，数据更新就压根不经过计算属性的逻辑。

举个例子，你写了个 `data() { return { price: 10 } }`，又写了个 `computed: { price() { return this.price * 2 } }`，这明显形成递归调用，浏览器直接栈溢出报错。



所以别折腾，命名保持唯一，让 data、computed、props 各司其职，Vue 才能正常追踪依赖和更新视图。

Vue Router 的跳转和 location.href 有什么区别？

Vue Router 的跳转不会刷新页面，它是在单页应用内部通过 **路由实例** 切换视图，而 `location.href` 是直接改变浏览器地址，会触发整个页面的重新加载。

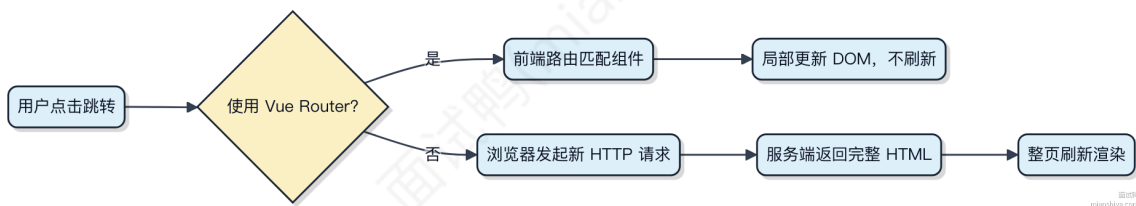
- 1) 用 Vue Router 跳转时，比如调用 `router.push('/home')`，框架会拦截这个操作，动态加载对应组件，更新 `router-view` 的内容，URL 变了但页面没刷新。整个过程由前端控制，体验更流畅，类似 App 的页面切换。
- 2) 而 `location.href = '/about'` 相当于在浏览器输入新地址回车，浏览器会向服务器发起全新请求，之前的 JavaScript 上下文全部丢失，所有资源重新下载解析，哪怕你用的是 Vue 写的 SPA，也会白屏一下再渲染。
- 3) 从数据层面看，前者能保留组件状态、Vuex 数据和页面滚动位置（可配置），后者一切归零。像在表单填写中途跳转，用 `location.href` 用户填的内容就没了。
- 4) SEO 场景下两者行为一致，因为现在搜索引擎都能执行 JS。但在服务端渲染（如 Nuxt.js）里，Vue Router 依然走客户端导航，避免不必要的 SSR 请求。

举个例子，你在 Element Plus 的后台系统里点菜单，用的是 `router-link`，背后就是 Vue Router；如果你写个按钮绑定 `location.href`，每次点击都会让 loading 动画重跑一遍，用户体验差很多。

代码上也很直观：

```
// 不会刷新页面
this.$router.push('/detail')

// 整页刷新，和 a 标签 href 一样
window.location.href = '/detail'
```



Vue Router 中 params 和 query 有什么区别？

params 和 query 都是 Vue Router 传参的方式，但用法和表现形式差别挺大。

query 传参就像 HTTP 的查询字符串，写在 URL 中 `?` 后面，刷新页面参数不会丢。比如 `/user?id=123`，通过 `this.$route.query.id` 取值。它适合传递非敏感、可暴露的筛选条件，像分页参数 `page`、`size` 这种。

params 是路径的一部分，需要路由配置里提前定义占位符。比如路由是 `/user/:id`，跳转时传 `/user/123`，通过 `this.$route.params.id` 取值。这种参数对用户不太透明，而且如果路由没配占位，压根不经过，参数就没了。

注意一点：params 在使用命名路由跳转时必须配合 `name` 字段，用 `path` 会失效。比如 `router.push({ name: 'user', params: { id: 123 } })` 才生效。

还有一个关键区别：**params 不会保留在浏览器历史记录的路径中**，如果你刷新页面，params 参数就丢了——除非你手动拼在路径里。而 query 刷新还在。

举个例子，管理后台查列表用 query，因为要支持分享链接；详情页用 params，路径语义清晰，也避免参数暴露。

active-class 是哪个 Vue 组件的属性？

`active-class` 是 Vue Router 中 `<router-link>` 组件的属性，用来配置当导航链接处于激活状态时，元素上应用的 CSS 类名。

默认情况下，`<router-link>` 在匹配到当前路由时会自动添加 `router-link-active` 类，如果希望自定义这个类名，就可以通过 `active-class` 来指定。比如在做高亮菜单或面包屑导航时，经常需要给当前页对应的链接加个高亮样式，这时候就用得上。

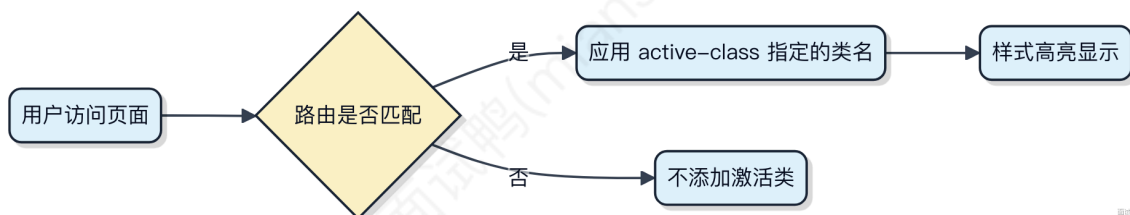
它还有一个兄弟属性 `exact-active-class`，用于精确匹配路由时的类名。

代码示例如下：

```
<router-link to="/home" active-class="active-menu">
  首页
</router-link>
```

当 `/home` 路由被激活时，生成的元素就会带有 `active-menu` 这个 class，而不是默认的 `router-link-active`。

注意这个属性只对 `<router-link>` 生效，普通的 Vue 组件没有这个特性。如果你用的是 `<a>` 标签或者自己封装的导航组件，就得手动判断 `$route` 的值来控制 class 了。



Vue 表单修饰符 .lazy 有什么作用？

输入框的默认行为是每次敲击键盘就同步数据，`.lazy` 修饰符的作用是把这个同步时机往后挪一挪。

- 1) 原生 input 事件在用户输入时频繁触发，`v-model` 会监听这个事件，导致 data 更新太勤快。对于性能敏感的场景或者不需要实时校验的情况，其实没必要这么高频更新。
- 2) 加上 `.lazy` 后，`v-model` 内部监听的事件从 `input` 变成了 `change`。也就是说，只有当输入框失去焦点或用户按下回车，才会把最新值同步到 data。
- 3) 典型使用场景比如搜索框防抖之前的简单优化，或者表单填写时不希望实时触发验证逻辑。配合 `.trim` 或 `.number` 一起用也常见。

```
<input v-model.lazy="searchText" />
```

这个改动看似小，但在输入频繁更新视图的场景下，能减少不少不必要的渲染开销。本质上是用响应延迟换取执行效率，属于典型的交互体验与性能之间的权衡。

Vue Router 中 route 和 router 有哪些区别？

`route` 和 `router` 听起来就差一个字母，但完全是两码事。

`router` 是整个路由系统的控制中心，你可以把它理解成导航的总指挥。我们调用 `router.push()` 或 `router.replace()` 都是通过它来发起页面跳转。它是通过 `createRouter` 创建出来的实例，掌管着所有路由规则和导航逻辑。

`route` 则是当前激活的路由信息，属于只读状态。它告诉你现在在哪个页面，比如当前路径、参数、查询字符串这些。这个对象通常在组件里通过 `useRoute()` 拿到，或者在路由守卫中作为参数传入。

打个比方，`router` 是手机上的地图 App，你能让它规划新路线；而 `route` 是当前显示的位置详情页，包含地址、POI 名称这些具体信息。

代码上也很直观：

```
import { useRouter, useRoute } from 'vue-router'

export default {
  setup() {
    const router = useRouter() // 拿到总控实例
    const route = useRoute()   // 拿到当前路由信息

    console.log(route.path) // 输出当前路径
    router.push('/home')    // 触发跳转
  }
}
```

常见的误区是把 `route.push` 当成跳转方法，这压根不成立。**router** 负责发号施令，**route** 只负责提供上下文数据。搞混这两个，在写导航守卫或动态跳转时很容易出错。

Vue 在 created 和 mounted 这两个生命周期钩子中请求数据有什么区别？

created 和 mounted 都能发请求，但时机不一样，带来的影响也不同。

created 阶段实例刚创建完，数据观测、属性和方法的初始化都完成了，这时候 DOM 还没生成，更别提挂载了。你在这里请求数据，可以尽早把网络调用发出去，不耽误事。比如用 axios 从后端拉用户信息，越早发越好，能缩短整体加载时间。

mounted 阶段就不同了，组件已经挂载到 DOM 上了，`$el` 可用，如果你的请求依赖 DOM 尺寸或位置（比如图表要根据容器宽度计算布局），就得放这。不过一般情况下，这种需求不多。

- 1) 如果只是单纯获取数据，推荐在 created 里发请求。早发起，页面看起来更快。
- 2) 如果需要操作 DOM 或依赖渲染结果，才放到 mounted。
- 3) 服务端渲染时注意，mounted 在 SSR 中不会执行，只有客户端激活才会触发，而 created 是 server 和 client 都会走。所以 SSR 场景下，数据请求必须放在 created，不然服务端拿不到数据。

代码层面没啥区别，都是调用接口：

```
created() {
  this.fetchData()
}
```

但语义和执行环境有差异。简单说，**数据请求尽量往前放**，不要卡在 mounted 等 DOM 挂载完成再动手，那纯属浪费时间。

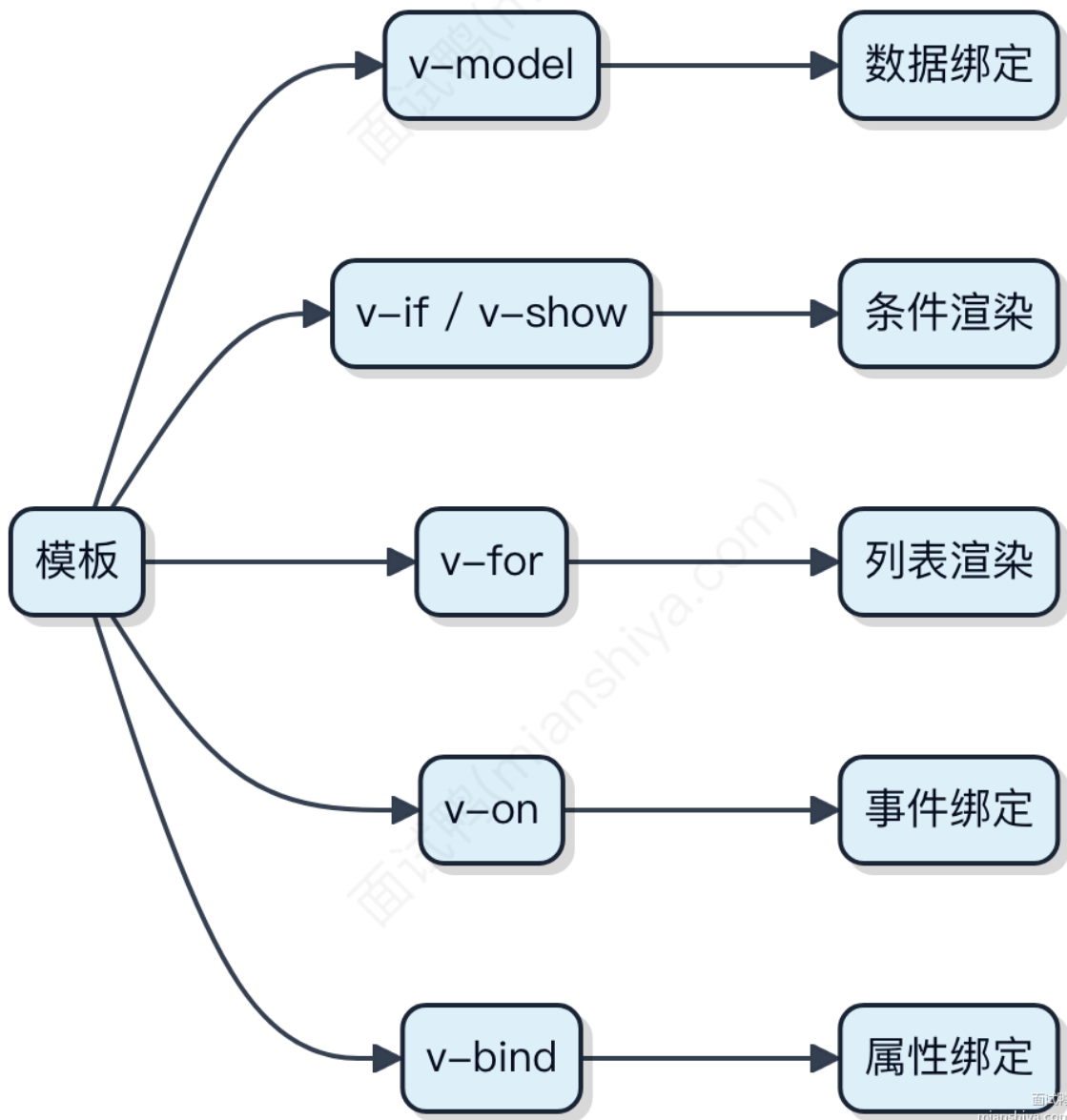
什么是 Vue 指令？Vue 有哪些常用的指令？

Vue 指令本质上是指带 `v-` 前缀的特殊属性，它告诉 Vue 在 DOM 上以某种方式应用副作用。比如数据变化时，指令会响应式地更新 DOM。

- 1) `v-model` 是双向绑定的核心，用在表单元素上，输入框内容和数据保持同步。写个搜索框基本就会用到它。
- 2) `v-if` 和 `v-show` 控制元素显隐，前者直接销毁或重建节点，后者切 `display` 样式。频繁切换选 `v-show`，条件复杂用 `v-if` 避免多余渲染。

- 3) `v-for` 遍历列表，配合 `key` 提升 diff 效率。别拿它遍历大数组，页面容易卡死。
- 4) `v-on` 绑定事件监听器，可以缩写成 `@click` 这种形式。事件处理函数里别干耗时操作，不然交互会卡顿。
- 5) `v-bind` 动态绑定属性，比如 `:class` 或 `:src`。静态属性直接写就行，动态值必须用 `v-bind`，不然 Vue 压根不经过响应式系统。
- 6) `v-once` 让节点只渲染一次，后续数据变了也不更新。适合渲染后就不变的内容，比如帮助文档标题。
- 7) `v-pre` 跳过编译过程，标签里的 `{{ msg }}` 会原样显示。调试模板或者展示代码片段时挺实用。

自定义指令也行，比如 `v-focus` 自动聚焦输入框，但大多数场景用组件更合适，指令更适合搞 DOM 操作这种脏活累活。



[为什么 Vue 的 data 属性必须声明为返回一个初始数据的函数？](#)

组件复用时，每个实例都需要一份独立的数据副本，如果 data 是共享的，修改一个实例的数据会影响其他所有实例。

1) Vue 组件可能被多次复用，比如页面里有 3 个 `<UserCard>` 组件。如果 data 是一个对象，那这 3 个组件会**共享同一个 data 对象**，改一个，全跟着变，这显然不行。

2) 函数每次执行都会返回一个**全新的对象**。把 data 定义成函数，每个组件实例调用它时都能拿到自己专属的一份数据，互不干扰。

```
data() {  
  return {  
    name: 'John',  
    age: 20  
  }  
}
```

3) 这个机制和 JavaScript 的闭包有关。Vue 在创建组件实例时会调用 data 函数，返回的数据对象会被纳入响应式系统。由于是函数作用域，每个实例的 data 都有自己的执行上下文，天然隔离。

简单说，就是为了**保证数据隔离**。组件是“类”，实例是“对象”，每个对象得有自己的状态，不能大家共用一套 data。

在 Vue 事件传入 \$event，使用 e.target 和 e.currentTarget 有什么区别？

`e.target` 和 `e.currentTarget` 都是 DOM 事件对象的属性，但它们指向的元素可能不一样。

1) `e.target` 是真正触发事件的那个元素。比如一个按钮内部有文字，你点的是文字，那 `e.target` 就是那个文本节点所在的标签，哪怕它是按钮里的 span 或 i 标签。

2) `e.currentTarget` 是绑定事件处理函数的那个元素。它永远是你 Vue 模板里写 `@click` 的那个父级元素，不管实际点击的是它的哪个子元素。

举个例子：你在 `<div @click="handler"> <span>点我</span> </div>` 上绑定了事件，你点的是 span，`e.target` 就是 span，而 `e.currentTarget` 是 div。

Vue 模板中常会这样用：

```
methods: {  
  handler(e) {  
    console.log(e.target); // 可能是子元素  
    console.log(e.currentTarget); // 一定是绑定事件的根元素  
  }  
}
```

所以判断事件来源时要小心。如果想确保操作的是绑定事件的元素，就得用 `e.currentTarget`，别被 `e.target` 带偏了。

有些场景下，比如事件委托，我们反而要靠 `e.target` 来判断具体点击了哪个子项。这种时候就靠它来做差异化处理。

介绍下 Vue 组件的命名规范？

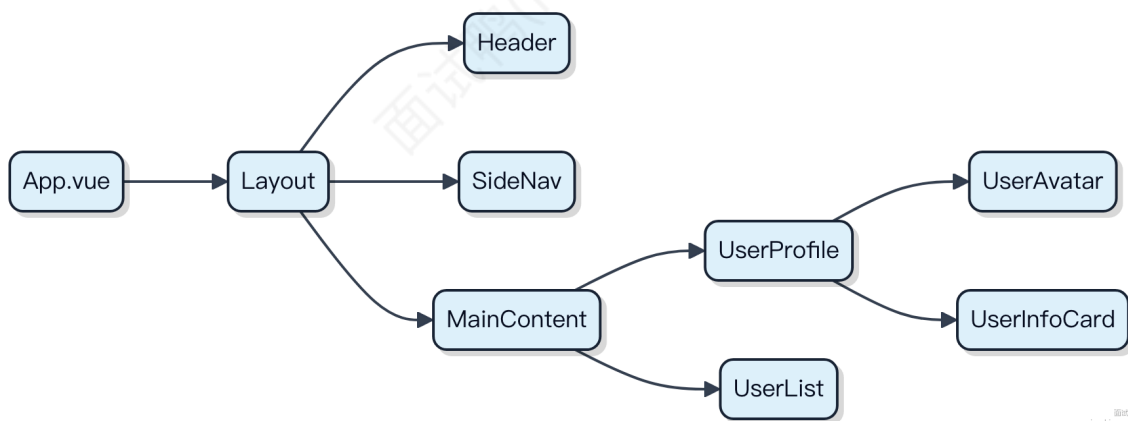
Vue 组件的命名直接影响项目的可维护性和团队协作效率，规范的核心是让组件名具备清晰的语义和一致的结构。

推荐使用 PascalCase（大驼峰）命名，比如 `UserProfile`、`DataChart`。这种写法在模板中能一眼识别出这是个自定义组件，和原生 HTML 标签区分开来。虽然 kebab-case（短横线分隔）如 `user-profile` 也合法，但在 JavaScript 对象里写字符串时容易出错，而且和 JS 的变量命名习惯不统一。

组件名要以 **单词组合** 表达功能，避免单个名词。比如用 `UserList` 而不是 `List`，用 `SearchInput` 而不是 `Input`。这样在嵌套多层时不容易混淆，也方便全局搜索定位。

如果项目规模大、模块多，可以加上模块前缀。例如 `AdminUserTable`、`ReportDownloadBtn`，避免不同模块间组件重名。

- 1) 根组件可以命名为 `App`，这是约定俗成的
- 2) 入口文件对应的组件通常叫 `Main` 或带 `Page` 后缀，比如 `HomePage`
- 3) 高阶组件或抽象封装，名称上体现行为，如 `WithLoading`、`ModalWrapper`



项目里所有组件保持统一风格，别一会大驼峰一会小写横杠，不然代码看起来像拼凑的。团队配合时，这条规则最好通过 ESLint 插件 `vue/multi-word-component-names` 强制约束。

如何访问 Vue 子组件的实例或子元素？

通过 `ref` 属性可以拿到子组件或子元素的实例。在模板里给目标元素或组件加上 `ref="xxx"`，然后在代码里用 `this.$refs.xxx` 访问。

1) 访问子组件实例时，`ref` 指向的是该组件的 Vue 实例，能直接调用它暴露的方法或访问数据。比如父组件要触发子表单的校验，就可以让子组件提供一个 `validate` 方法，父组件通过 `this.$refs.formRef.validate()` 来调。

2) 如果是原生元素，比如 `<input ref="inputRef">`，`this.$refs.inputRef` 拿到的就是 DOM 节点，可以直接调用 `focus()` 或操作属性。

注意 `ref` 只在组件渲染完成后才可用，**初始渲染时为 null**，一般在 `mounted` 生命周期之后才能安全使用。同时避免在模板中过度依赖 `$refs`，容易破坏组件封装性。

父子组件通信优先用 `props` 和事件，`ref` 更适合命令式操作，比如手动 `focus`、滚动位置控制、触发动画这类场景。

如何重置 Vue 的 data？

要重置 Vue 实例的 data，本质是把当前组件的数据恢复到初始声明时的状态。Vue 本身没有提供直接的 reset API，但有几种实用方式。

最常见的是通过调用组件实例的 `$data` 并结合 `this.$options.data()` 来实现。因为组件的 data 选项在定义时是一个函数，返回初始数据对象，所以你可以再次执行它拿到原始值。

```
// 在组件方法中
resetData() {
  this.$data = this.$options.data.call(this)
}
```

这种方式适用于 Options API。注意，`$options.data` 必须用 `call(this)` 调用，确保内部的 `this` 指向正确，尤其是在使用依赖注入或 mixin 的场景下。

如果用了 Composition API（尤其是 `<script setup>`），情况略有不同。由于 setup 中没有 `this`，通常会借助 `ref` 或 `reactive` 手动维护状态，并提供一个重置函数。

```
import { ref } from 'vue'

const state = ref({
  name: '',
  count: 0
})

const resetState = () => {
  state.value = { name: '', count: 0 }
}
```

这种模式更灵活，也更容易测试和管理。

- 1) 不要直接修改 `$data` 外部状态而不触发响应式更新
- 2) 避免递归重置复杂嵌套对象，可能引发性能问题
- 3) 如果用了 Vuex/Pinia，记得区分局部组件状态和全局状态，重置逻辑应分层处理

对于高频率使用的表单组件，建议封装一个 `useReset` 自定义 Hook，统一管理重置行为。

Vue 中怎么获取 DOM 节点？

通过 `ref` 选项可以获取组件或 DOM 节点的实例。在模板中给元素设置 `ref` 属性后，就能在 JavaScript 中通过 `this.$refs` 访问到它。

- 1) 操作原生 DOM 元素

比如想手动聚焦一个输入框，可以直接拿到 input 的 DOM 节点调用 `focus` 方法。

```
<template>
  <input ref="inputRef" />
</template>

<script>
export default {
  mounted() {
```

```
    this.$refs.inputRef.focus()
  }
}
</script>
```

2) 访问子组件实例

ref 不只能拿 DOM，还能拿到子组件的实例，进而调用它的方法或访问数据。

```
<template>
  <ChildComponent ref="child" />
</template>

<script>
export default {
  methods: {
    callChildMethod() {
      this.$refs.child.someMethod()
    }
  }
}
</script>
```

注意 ref 是**非响应式**的，只在渲染完成后存在。如果在 v-for 里使用 ref，得到的是一个数组或对象集合。

另外 Composition API 里用法稍有不同，要用 `template refs` 配合 `setup` 和 `onMounted` 使用。

```
<template>
  <div ref="root">Hello</div>
</template>

<script>
import { ref, onMounted } from 'vue'
export default {
  setup() {
    const root = ref(null)
    onMounted(() => {
      console.log(root.value) // DOM 节点
    })
    return { root }
  }
}
</script>
```

Vue 使用 v-for 遍历对象时，是按什么顺序遍历的？如何保证遍历顺序？

遍历对象时的顺序取决于 JavaScript 自身的枚举顺序，Vue 不会额外干预。现代浏览器中，普通对象的 key 遍历遵循这样的优先级：首先是数字类型的 key（按升序），然后是字符串类型的 key（按定义时的先后顺序），最后是 Symbol 类型的 key。

也就是说，如果你用 v-for 遍历一个对象：

```
<li v-for="(value, key) in obj" :key="key">
  {{ key }}: {{ value }}
</li>
```

它的输出顺序不是随机的，但你也**不能完全依赖**它在所有环境一致，尤其是当对象动态增删属性时。比如：

```
obj = { 2: 'b', 1: 'a', c: 'c', a: 'a' }
```

实际渲染顺序会是 1, 2, c, a —— 数字 key 先排序，字符串 key 按声明顺序。

想要**保证顺序**，最可靠的方式是不直接遍历对象，而是用数组明确控制顺序。比如配合 `Object.keys()` 或定义一个 key 的数组：

```
<li v-for="key in Object.keys(obj).sort()" :key="key">
  {{ key }}: {{ obj[key] }}
</li>
```

或者更稳妥的做法是使用数组来维护顺序：

```
data() {
  return {
    order: ['name', 'age', 'city'],
    obj: { name: 'Alice', age: 30, city: 'Beijing' }
  }
}
```

```
<li v-for="key in order" :key="key">
  {{ key }}: {{ obj[key] }}
</li>
```

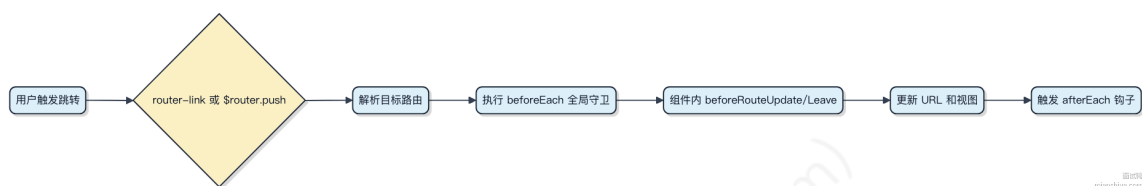
这样一来，遍历顺序完全由你掌控，不再受 JS 引擎对象内部实现的影响。

Vue 路由之间是如何跳转的？有哪些跳转方式？

路由跳转本质上是 URL 变化触发视图更新的过程。Vue Router 通过监听路由变化，动态渲染对应组件。

- 1) 声明式导航用 `router-link`，模板里写 `<router-link to="/home">Home</router-link>`，点击自动跳转，底层其实是调用了编程式方法。
- 2) 编程式导航直接操作路由实例。 `this.$router.push('/home')` 或 `this.$router.push({ name: 'user', params: { id: 88 } })`，会新增一条历史记录。想替换当前记录不留下痕迹，就用 `replace`，比如登录后跳首页不想让用户点返回再回登录页。
- 3) 前进后退用 `go(n)`，`back()` 等价于 `go(-1)`，适合从详情页返回列表时保持原位置。

跳转前通常要做权限判断。导航守卫 `beforeEach` 拦一下，比如未登录就重定向到登录页。异步场景比如表单未保存，可以在组件内定义 `beforeRouteLeave`，弹确认框阻止跳转。



一般 push 最常用，replace 用于无痕跳转，go 控制浏览历史。守卫机制让跳转过程可控制，权限、数据校验都能卡住。

什么是 Vue 的前端路由？如何实现？

前端路由的核心是不刷新页面的情况下实现视图的切换，靠的是监听 URL 中 hash 或者 history 的变化。

1) 基于 hash 模式

URL 中 # 后的部分改变不会触发页面刷新，通过监听 hashchange 事件就能拿到路由变化。Vue Router 默认用的就是这种模式，兼容性好，IE8 都能跑。

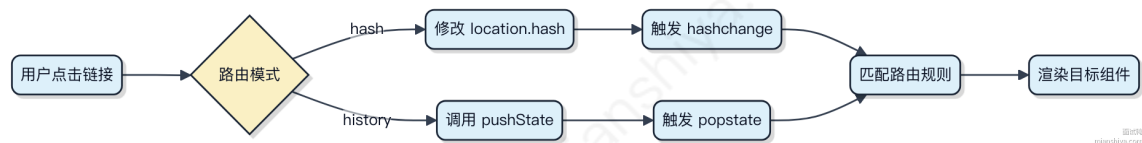
```

window.addEventListener('hashchange', () => {
  const path = window.location.hash.slice(1) || '/'
  // 匹配路由，渲染对应组件
})
  
```

2) 基于 history 模式

用 pushState 和 replaceState 修改 URL，配合 popstate 监听前进后退。URL 干净，但需要服务端支持，否则用户直连或刷新会 404。Nginx 要配置 fallback 到 index.html。

Vue Router 把路由映射和组件解耦，通过声明式 <router-link to="/"> 和编程式 this.\$router.push('/') 触发跳转，内部维护当前状态，触发对应的组件更新。



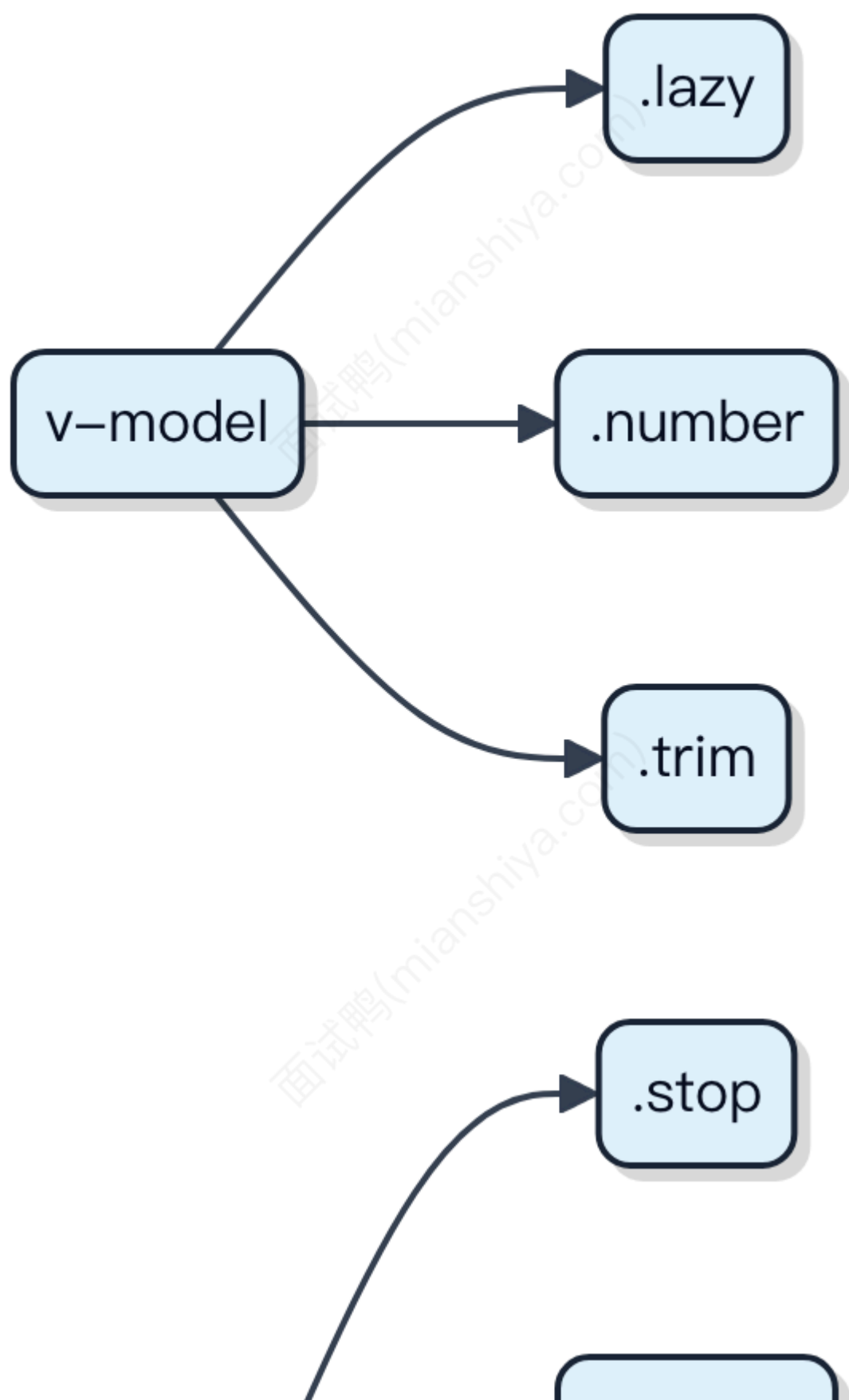
一般小项目用 hash，简单省事；中大型项目上 history，体验更接近原生应用，但得搭好服务端兜底。

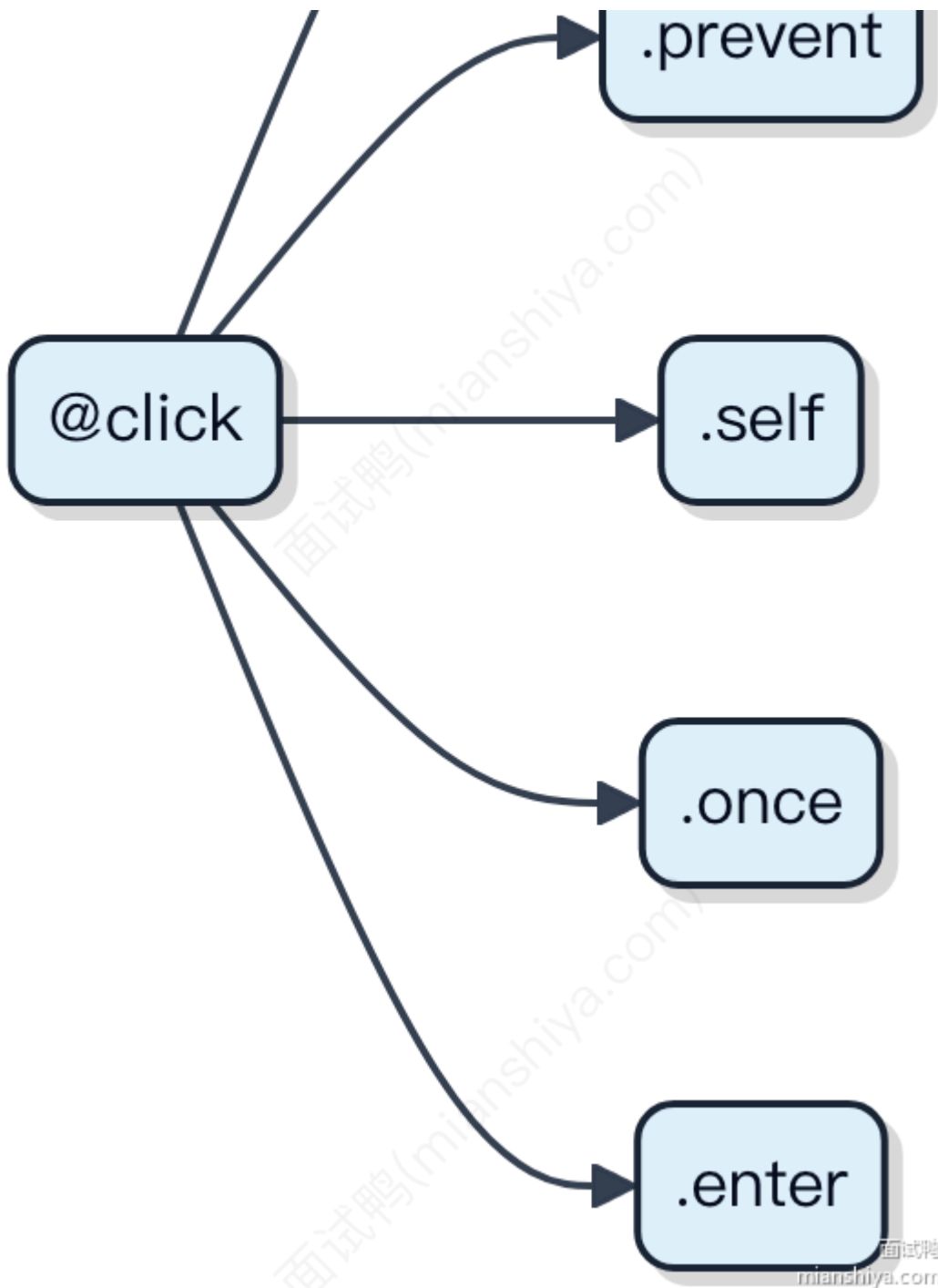
Vue 中有哪些表单修饰符和事件修饰符？

表单修饰符主要解决数据绑定时的常见需求。比如你用 v-model 绑定一个输入框，加上 .lazy 就不会在每次敲击键盘时同步，而是等失去焦点再更新，避免频繁触发计算。.number 比较实用，用户输入的数字哪怕看起来是字符串，也会自动转成 Number 类型，防止后续计算出问题。还有 .trim，专门处理首尾空格，适合用户名、邮箱这类不能带空格的字段。

事件修饰符用来简化 DOM 事件的默认行为控制。比如 .stop 阻止冒泡，.prevent 拦截默认提交动作，写表单时特别常见。.self 只有点击元素本身才触发，能避免捕获子元素的事件。按键修饰符像 .enter，可以限定只在按下回车时触发搜索操作。.once 保证事件只响应一次，适合弹窗关闭或初始化操作。

这些修饰符可以直接连在指令后面，代码看起来干净，不用在方法里写一堆 event.preventDefault() 这种脏活累活。





组合使用也很灵活，比如 `@click.stop.prevent` 同时阻止冒泡和默认行为。多个修饰符按顺序从左到右执行，逻辑清晰。

Vue 的生命周期总共有哪几个阶段？

Vue 实例从创建到销毁会经历多个阶段，每个阶段都有对应的钩子函数可以注入逻辑。

1) beforeCreate

实例刚被创建，数据观测和事件配置还没开始，`data`、`methods` 还没初始化，访问不到。

2) created

数据已经挂载，`data`、`methods` 可以访问，一般做数据初始化、发起异步请求。但 DOM 还没生成，`$el` 不存在。

3) beforeMount

模板编译完成，即将开始挂载。服务端渲染不会走这个钩子。

4) mounted

DOM 已经挂载到页面，可以操作真实节点。组件完全可用，常见于操作 DOM 或开启轮询。

5) beforeUpdate

响应式数据变更触发虚拟 DOM 重新渲染前执行，适合在更新前做一些处理。

6) updated

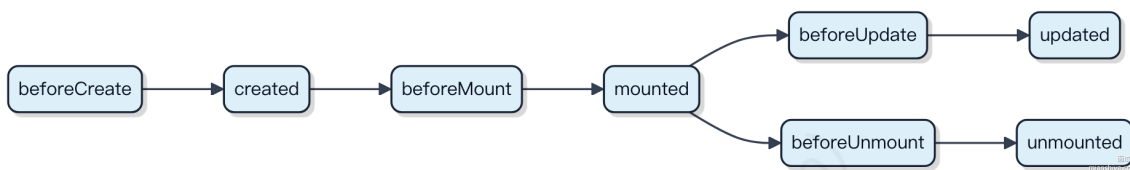
视图更新完毕，注意这里可能因为子组件更新多次触发，避免在此发请求造成循环。

7) beforeUnmount (Vue 3) / beforeDestroy (Vue 2)

实例销毁前，解绑事件、清除定时器、移除监听都在这一步做，防止内存泄漏。

8) unmounted / destroyed

组件已完全销毁，所有绑定和子实例都被移除，不能再访问实例。



实际开发中，**created** 和 **mounted** 是最常用的两个阶段。异步数据拉取一般放在 **created**，涉及 DOM 操作则放 **mounted**。

你了解 Vue 中的过滤器吗？它有哪些应用场景？

Vue 的过滤器本质上是一个函数，用来对模板中的文本进行格式化处理。它只能用在双大括号插值和 **v-bind** 表达式中，接收一个值作为输入，返回处理后的结果。

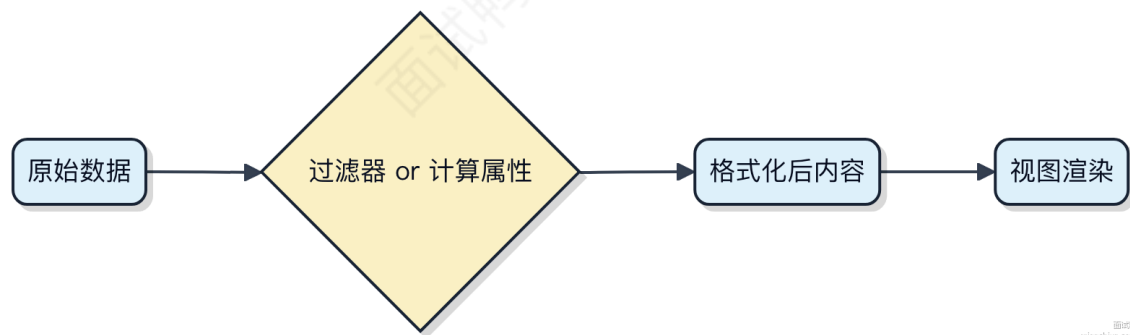
- 1) 定义方式支持局部和全局两种。全局过滤器通过 `Vue.filter('formatName', fn)` 注册，组件内直接使用；局部则在组件选项里定义 `filters` 对象。
- 2) 常见应用场景比如时间戳转日期、数字添加千分位、金额补上单位“元”、字符串截取显示省略号等。这类操作不改变原始数据，只影响视图展示。
- 3) 但要注意，从 Vue 3 开始过滤器已经被移除，官方推荐用方法调用或计算属性替代。因为过滤器逻辑隐含在模板中，调试和复用都不如函数直观。

代码示例如下：

```
// 全局注册
Vue.filter('currency', function (value) {
  if (!value) return ''
  return (value / 100).toFixed(2) + '元'
})
```

模板中使用：`{{ price | currency }}`。这种方式写起来确实简洁，但团队项目里容易积累一堆难以追踪的“黑盒”处理函数。

其实更推荐把格式化逻辑放在计算属性或组合式函数（composable）里，结构清晰也方便测试。尤其在复杂业务场景下，**可维护性**比少写几行代码重要得多。



所以总结一句话：能用计算属性或方法的地方，就别依赖过滤器，尤其是新项目。

Vue 中 data 的属性可以与 methods 中的方法同名吗？为什么？

Vue 实例上的所有属性和方法最终都会被代理到同一个对象上，所以命名冲突会直接导致覆盖或异常。

1) data 里的属性在初始化时会被 Vue 响应式系统处理，挂载到实例的 `_data` 上，并通过 `Object.defineProperty` 或 `Proxy` 代理到 `vm` 实例上，让你能用 `this.xxx` 直接访问。

2) methods 里的方法也会被代理到 `vm` 实例上，同样是 `this.methodName` 就能调。如果 data 中某个属性和 method 同名，后定义的会覆盖前一个。

Vue 在合并选项时，methods 是在 data 之后处理的。假设你在 data 里定义了 `log`，然后 methods 里也有 `log`，那 `methods.log` 会覆盖 data 的 `log` 属性。反过来，如果 data 的 `log` 在后面（比如通过 `mixin` 注入），它反而会把 methods 的方法顶掉，这时候调用 `this.log()` 就会报错 “is not a function”。

这种覆盖行为不是响应式的锅，而是 Vue 实例属性合并机制决定的。官方文档也明确说：**不要让 data、methods、computed 里的名字重复。**

```
new Vue({
  data() {
    return {
      name: 'vue',
      log: 'hello' // ❌ 别这么干
    }
  },
  methods: {
    log() { console.log(this.name) } // 被 data 的 log 覆盖
  }
})
```

运行时调 `this.log()` 就会出问题。简单说，同名就会打架，谁后处理谁说了算，但结果基本都是 bug。

如何定义 Vue 的动态路由？如何获取传过来的动态参数？

动态路由在 Vue 里其实就是通过 `path` 中的参数占位来实现的。比如你有个用户详情页，每个用户的 ID 不同，不可能为每个 ID 写一个路由，这时候就用动态段。

你在定义路由的时候，在 `path` 里用冒号 `:` 标记参数名：

```
const routes = [
  { path: '/user/:id', component: UserComponent }
]
```

这样 `/user/123` 和 `/user/456` 都会匹配到同一个路由，`123` 或 `456` 就是动态参数。

组件里怎么拿这个 `id`？通过 `this.$route.params.id` 就能拿到。注意是 `$route`，不是 `router`，很多人在这搞混。

如果路由是 `/user/:id/post/:postId`，那 `params` 里就有两个字段：`id` 和 `postId`。

这种模式特别适合内容页、详情页这类 URL 结构固定但 ID 变化的场景。像后台管理系统里的订单详情、商品编辑都这么干。

需要提醒的是，**参数变化不会触发组件重新创建**，也就是说如果你在同一个路由下从 `/user/1` 跳到 `/user/2`，`beforeRouteUpdate` 守卫或者监听 `$route` 的变化才能捕获到。

Vue Router 有什么作用？它有哪些组件？

Vue Router 是 Vue.js 的官方路由管理器，它让构建单页面应用（SPA）变得自然。页面切换时不需要重新加载整个页面，而是根据当前 URL 动态渲染对应的组件，用户体验更流畅。

路由配置一般是一个数组，定义路径和对应组件的映射关系。比如访问 `/user/123`，可以渲染 `UserComponent` 并把 `123` 作为参数传进去。这种模式在中后台系统、管理平台里特别常见，像 Element Plus 搭配 Vue Router 就是典型组合。

它提供的核心组件有两个：

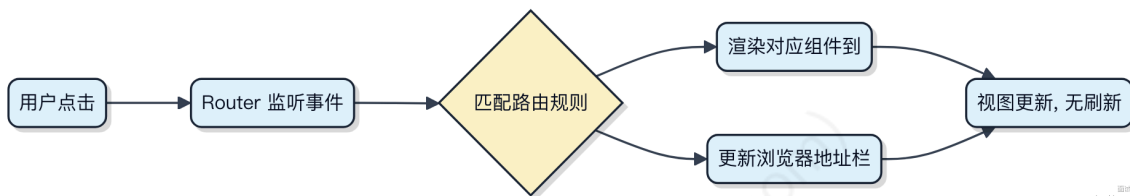
1) `<router-link>`

用来替代原生 `<a>` 标签实现导航。点击时不会发起全页请求，而是通过 JavaScript 更新视图和浏览器历史记录。默认渲染成带 `href` 的 `<a>`，但**压根不经过网络请求**。

2) `<router-view>`

占位组件，路由匹配到的组件会渲染在这里。你可以把它看作是“页面内容的出口”，每次路由变化，它决定该显示哪个组件。

路由跳转分两种方式：声明式用 `<router-link to="/xxx">`，编程式调用 `router.push('/xxx')`。后者适合在登录成功后跳转到首页这类逻辑判断场景。



注意别把 `<router-link>` 当按钮用，复杂交互还是推荐 `router.push` 控制更灵活。

在 Vue 中使用 this 时应该注意哪些问题？

`this` 的指向在 Vue 选项式 API 里基本是固定的，它指向当前组件实例。但你在写 `methods`、`watch` 或生命周期钩子时，如果用了箭头函数，就得小心了。

1) 别在 `methods` 里用箭头函数。因为箭头函数绑定了外层作用域的 `this`，你调用 `this.data` 或 `this.methods` 里的其他方法时，可能压根拿不到组件实例，数据响应就断了。

2) `watch` 回调如果用箭头函数，`this` 拿不到 `vm` 实例，想访问 `data` 或 `methods` 就会搞不定。官方推荐用普通函数，保证 `this` 正确指向。

3) 在 `setup` 或组合式 API 中，`this` 压根不指向组件实例。`setup` 执行时组件还没创建，所以不要依赖 `this`。状态和方法都得通过 `ref`、`reactive` 和 `return` 出去，用解构或 `context` 参数替代。

4) 事件回调里如果用了 `this`，绑定到 DOM 时要确保函数没有脱离实例上下文。比如把 `methods` 里的函数直接赋给 `setTimeout` 或 `addEventListener`，得手动 `bind(this)`，不然 `this` 会变成 `window` 或 `undefined`。

```

methods: {
  handleClick() {
    // 正常使用 this
    console.log(this.message)
  },
  badUse() {
    setTimeout(() => {
      // 箭头函数没问题，能访问 this
      console.log(this.message)
    }, 100)

    setTimeout(function () {
      // 普通 function, this 已丢失
      console.log(this.message) // undefined
    }, 100)
  }
}

```

总的来说，选项式 API 里保持用普通函数，组合式 API 里干脆忘了 this，用 ref 和 reactive 管理状态更稳妥。

什么是 Vue 中的 slot？它有什么作用？

Vue 里的 slot 其实就是组件的内容分发机制。你写一个组件时，不可能把所有内容都固定死，总得留点地方让父组件来决定放啥。

比如你封装了一个卡片组件 `<Card>`，标题、内容、底部操作区都想让使用者自己定义。这时候就用 slot，它相当于在组件模板里开了几个“插槽口”，等着别人把内容塞进来。

默认情况下，父组件传入的额外内容会直接被丢进默认插槽。如果你想更灵活，可以用具名插槽。比如在 `Card` 组件里：

```

<template>
  <div class="card">
    <header><slot name="header"></slot></header>
    <main><slot></slot></main>
    <footer><slot name="footer"></slot></footer>
  </div>
</template>

```

父组件就可以这样用：

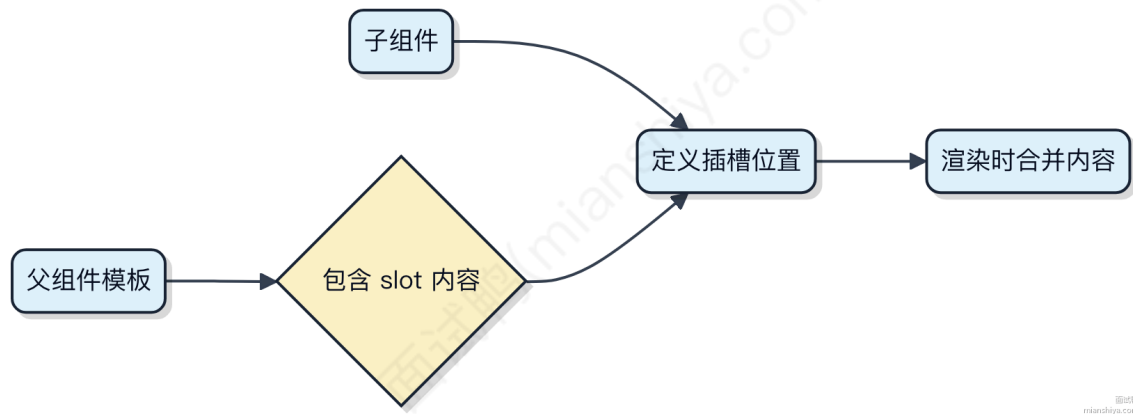
```

<Card>
  <template #header>这里是标题</template>
  <p>这是主体内容</p>
  <template #footer>
    <button>取消</button>
    <button>确定</button>
  </template>
</Card>

```

注意这里的 `#header` 就是 `v-slot:header` 的缩写。**作用域插槽**还能把子组件的数据传给父组件，让父组件根据这些数据来渲染内容。比如列表组件把 `item` 数据通过 `slot props` 抛出去，外面就能自定义每一项长啥样。

整个过程压根不经过 `prop` 传递，而是模板层面的内容注入，特别适合做高可定制的 UI 组件库，像 `Element Plus` 这些都是靠 `slot` 实现灵活布局的。



在 Vue 组件中如何访问根实例？

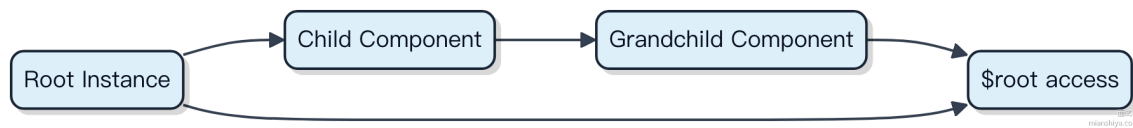
Vue 组件里要访问根实例，最直接的方式是通过 `$root`。每个组件实例都有这个属性，指向应用的根实例。比如在嵌套好几层的子组件里，想拿根实例上的数据，可以直接用 `this.$root.someData`。

1) `$root` 获取的是根组件实例，不管当前组件嵌套多深都能拿到。2) 如果是 Vue 3 的 `<script setup>` 语法，可以通过 `getCurrentInstance` 拿到上下文，然后访问 `ctx.root`。3) 但 `$root` 属于直接依赖根实例，耦合度高，一般建议只读不改，避免脏活累活都堆到根上。

```
// Vue 2 Options API
export default {
  mounted() {
    console.log(this.$root)
  }
}

// Vue 3 Composition API
import { getCurrentInstance } from 'vue'
setup() {
  const instance = getCurrentInstance()
  console.log(instance.ctx.root)
}
```

需要跨层级通信时，更好的做法是用 `provide / inject` 或状态管理如 `Pinia`，而不是层层传递 `props` 或滥用 `$root`。尤其在大型项目里，过度依赖 `$root` 会让组件难以维护和测试。



Vue 的 v-cloak 和 v-pre 指令有什么作用？

Vue 在首次渲染时，如果模板里有 Mustache 语法，比如 {{ message }}，浏览器会先看到这些原始标签，等 Vue 实例挂载完成后才会替换。这就可能造成一瞬间的“内容闪烁”。

v-cloak 就是干这个的。你只要在元素上写上 v-cloak，Vue 初始化完之前，这个属性一直挂着。你可以配合 CSS 把带 v-cloak 的元素先隐藏掉，等 Vue 接管后自动消失。

```
[v-cloak] {  
  display: none;  
}
```

这样页面加载时就不会看到乱七八糟的 {{}} 了，体验更顺。

另一个是 v-pre，它的作用是跳过这个节点和它的子节点的编译过程。说白了，就是告诉 Vue：这块你别管，我不需要响应式更新。适合用在纯展示的静态内容上，比如文档页面里大段的代码示例，能减少不必要的解析开销，提升一点性能。

- 1) v-cloak 配合 CSS 解决首次渲染的闪烁问题
- 2) v-pre 跳过编译，用于静态内容提升性能

这两个都不是高频指令，但在特定场景下能帮你把细节处理得更干净。

Vue Router 中如何获取路由传递过来的参数？

路由参数的获取方式主要看参数类型，Vue Router 支持三种：params、query 和路由组件的 props。

- 1) 动态路径参数用 params，比如 /user/:id，在组件内通过 this.\$route.params.id 拿。注意刷新页面时 params 不会丢失，因为它是路径的一部分。
- 2) 查询参数走 query，像 /search?keyword=vue，取值是 this.\$route.query.keyword。这种适合可选、非必填的参数，传参灵活但不会影响路由匹配。
- 3) 推荐把路由参数转为组件 props 传入，解耦更清晰。需要在路由配置里开启 props: true，组件就能直接用 props 接收，测试和复用都方便。

```
// 路由配置  
{  
  path: '/user/:id',  
  component: User,  
  props: true  
}  
  
// 组件  
export default {  
  props: ['id'],  
  mounted() {  
    console.log(this.id) // 直接用，不用再从 $route 拿  
  }  
}
```

别忘了 `params` 只能用 `name` 跳转才能保留, 用 `path` 的话会丢掉。 `query` 就没这限制, 用 `path` 或 `name` 都行。

Vue 中是如何使用 event 对象的?

在 Vue 的模板语法里, 事件处理函数默认接收原生 DOM 的 event 对象, 和原生写法一样可以直接用。比如你绑个 `@click="handleClick"`, 方法里就能直接拿到 `event` 参数。

如果方法需要额外参数, 又想访问 `event`, 可以用 `$event` 显式传入:

```
<button @click="say('hello', $event)">Submit</button>
```

```
methods: {  
  say(message, event) {  
    if (event) event.preventDefault()  
    console.log(message)  
  }  
}
```

Vue 对所有浏览器做了 event 兼容处理, 你拿到的就是标准化后的对象, 不用担心 IE 的老问题。不过要注意, 在 `<template>` 中的事件是 DOM 事件, 不是自定义事件, 自定义事件要配合 `$emit` 在组件间通信时使用。

另外, 修饰符能帮你简化 event 常见操作。例如 `.stop` 阻止冒泡、`.prevent` 阻止默认行为、`.once` 只触发一次, 写起来比在方法里手动调 `stopPropagation()` 干净多了。

```
<!-- 阻止点击冒泡 -->  
<a @click.stop="doThis"></a>  
  
<!-- 提交事件不再重载页面 -->  
<form @submit.prevent="onSubmit"></form>  
  
<!-- 一次性监听 -->  
<button @click.once="init">Only once</button>
```

这些修饰符可以链式使用, Vue 内部会生成对应的包装逻辑, 代码更声明式, 也更容易读。

Vue Router 如何配置 404 页面?

路由没匹配上时, 用户访问不存在的路径, 得有个兜底页面, 404 页面就是干这个用的。Vue Router 里配起来很简单, 关键在于**顺序和通配符路由**。

定义一个 catch-all 路由, 路径写成 `/*` 或 `/404`, 然后把它放到路由数组的最后。因为 Vue Router 是从上往下匹配的, 只要前面的都没中, 就会落到这个兜底路由上。

```
const routes = [  
  { path: '/', component: Home },  
  { path: '/about', component: About },  
  // 其他正常路由...  
  { path: '/*', component: NotFound }  
]
```

```
{ path: '/*', component: NotFound }
]
```

这里用了动态参数 `pathMatch` 捕获整个路径段，传给组件还能展示用户想找的地址。NotFound 组件里可以提示“页面走丢了”，加个返回首页的按钮。

- 1) 必须放最后，不然会拦截所有路由
- 2) `path: '/*'` 写法在某些版本不推荐，官方建议用 `/:pathMatch(.)*`
- 3) 服务端要配合返回 404 状态码，单页应用前端路由只是体验优化

如果项目有 SSR，比如 Nuxt.js，还得在服务端配置对应的 404 响应，不然搜索引擎看不到真正的 404 状态。

Vue 的 template 标签有什么用？

`template` 标签在 Vue 里不是用来直接渲染的，它本身不会出现在最终的 DOM 结构里。它的作用是定义一块内容模板，这块模板什么时候渲染、怎么渲染，由其他指令来控制。

比如用 `v-if` 或 `v-for` 的时候，你可能需要包裹多个元素，又不想额外加一个 `div` 影响结构。这时候就把这些元素包在 `<template>` 里，Vue 会解析里面的内容，但不会把它自己渲染成一个节点。

```
<template v-if="show">
  <h1>标题</h1>
  <p>一段说明</p>
  <button>操作</button>
</template>
```

上面这段代码，如果 `show` 为 `true`，三个子元素都会插入 DOM，但压根不经过 `<template>` 节点。

另一个常见场景是配合 `v-slot` 使用，定义插槽内容。父组件传过来的模板片段，就是通过 `<template>` 包裹并指定作用域的。

```
<MyComponent>
  <template #header="slotProps">
    <h2>{{ slotProps.title }}</h2>
  </template>
</MyComponent>
```

这里 `#header` 定义了一个具名插槽，`template` 提供了作用域变量的接收位置。组件内部用 `slotProps` 把数据传出来，外面就能用了。

总的来说，`template` 就是个逻辑容器，干的是脏活累活，自己却隐身不见。没有它，很多结构控制和插槽机制都搞不定。

为什么 Vue 中的 data 属性是一个函数而不是一个对象？

Vue 组件可能被多次复用，每次复用都应该拥有自己独立的数据副本。如果 `data` 是一个对象，那所有组件实例都会共享这同一个对象的引用。

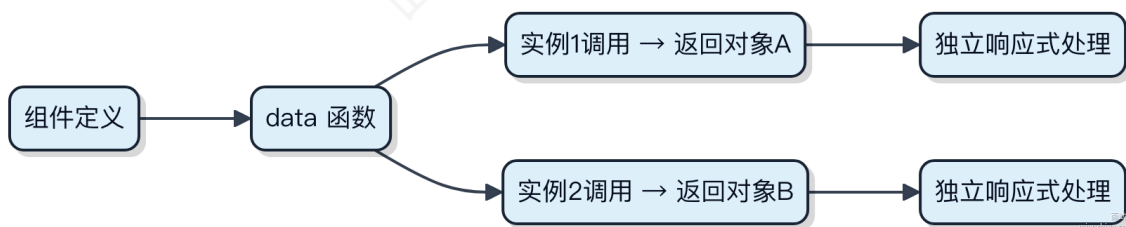
想象一下，10 个相同的组件都指向同一个 `data` 对象，其中一个改了数据，其他 9 个全跟着变，这显然不是我们想要的。

所以 Vue 要求 data 必须是一个函数，每次创建实例时，都通过调用这个函数返回一个**全新的对象**，这样每个实例就有了独立的 data 空间，互不干扰。

```
// 正确写法：data 是函数，返回新对象
data() {
  return {
    count: 0,
    list: []
  }
}
```

你可能会在根实例里看到直接用对象形式的 data，比如 `new Vue({ data: {} })`，那是因为根实例只存在一个，不存在复用和数据污染问题，属于特例。

至于组件选项中的其他属性，比如 `methods`、`computed`，它们本身是方法或访问器，天然按实例绑定，不存在状态共享的问题，所以不需要用函数返回。



在 Vue 渲染模板时，如何保留模板中的 HTML 注释？

Vue 默认在生产环境下会移除模板中的 HTML 注释，但在某些场景下我们可能需要保留它们，比如用于标记位置、调试或配合一些依赖注释的第三方库。

控制这个行为的关键是 `comments` 选项。在创建 Vue 应用实例时，可以通过配置 `comments: true` 来强制保留模板中的注释。

```
const app = createApp(App)
app.mount('#app', true) // 第二个参数是 `hydrate`，和注释无关

// 想要保留注释，需要在挂载时传入配置
app.mount('#app', {
  comments: true
})
```

这个 `comments` 配置只在使用完整版 Vue（包含模板编译器）时才生效。如果你用的是 `runtime-only` 版本，模板已经在构建阶段编译成 `render` 函数，那这个选项就不起作用了。

开发环境下一般问题不大，但生产环境为了性能，默认会把注释干掉。毕竟 DOM 节点越多，内存开销越大。保留注释意味着每个 `<!-- -->` 都会变成一个真实的 `Text Node`，浏览器得为它分配内存。

所以要不要保留，得权衡清楚。像 `v-if` 的边界、`v-for` 的锚点这些，Vue 内部其实是用注释节点做标记的，但那是框架自己用的，和我们写的模板注释不是一回事。

一般来说，除非有明确需求，比如静态站点生成器需要保留特定标记，或者你在做 SSR 并依赖注释做 hydration 定位，否则没必要开启。

为什么不建议在 Vue 中同时使用 v-if 和 v-for?

Vue 在处理指令时会按照优先级顺序解析，**v-for 的优先级高于 v-if**。这意味着即使你把两个指令写在同一个元素上，Vue 也会先执行 v-for 进行列表渲染，然后再对每一项单独判断 v-if 的条件。

这就带来两个明显问题。第一，性能浪费。v-if 是基于条件控制显示或隐藏，但如果它和 v-for 一起用，即便某项数据不符合 v-if 条件、最终不会显示，它依然会被遍历、创建虚拟 DOM、执行判断逻辑。比如有 100 条数据，只有 10 条满足条件，那剩下的 90 项压根不经过渲染，但它们的循环开销是实打实存在的。

第二，代码可读性差。这种写法容易让人误解为“只遍历符合条件的项”，但实际上它是“遍历所有项再逐个过滤”，语义不清晰，后期维护容易出错。

正确的做法是，把 v-if 写在外层包裹元素上，控制整个列表区域的显隐。如果需要条件性渲染部分数据，应该在 computed 中预先过滤好数据源，让 v-for 只处理精简后的数组。

```
// 推荐：计算属性预处理
computed: {
  activeItems() {
    return this.items.filter(item => !item.isInactive)
  }
}
```

或者结构拆分：

```
<ul v-if="items.length">
  <li v-for="item in items" :key="item.id">{{ item.name }}</li>
</ul>
<p v-else>暂无数据</p>
```

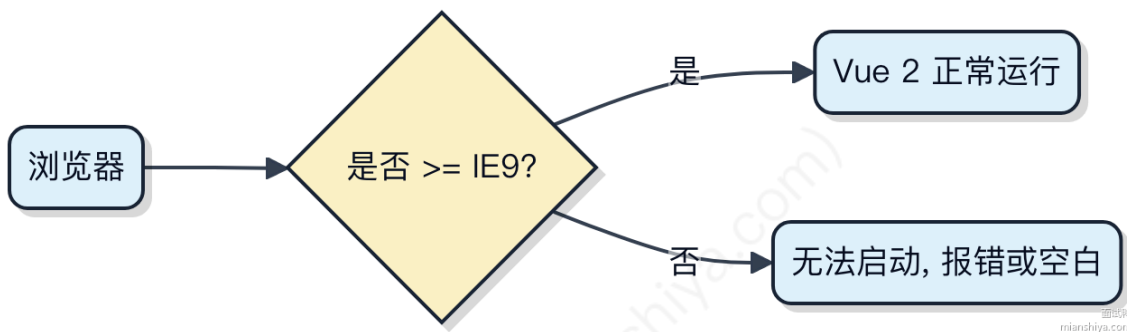
Vue 2.0 支持哪个版本以上的 IE 浏览器?

Vue 2.0 官方支持 IE9 及以上版本。IE8 及更低版本无法运行 Vue 2 的任何功能，因为 Vue 2 依赖的 ECMAScript 5 特性（比如 `Object.defineProperty`）在 IE8 及以下压根不被支持。

Vue 2 在初始化数据劫持时，会通过 `Object.defineProperty` 对 data 做响应式处理，而这个 API 在 IE8 中只支持 DOM 对象，不支持普通 JS 对象，这就导致 Vue 根本无法建立响应式系统。

如果非得兼容 IE8，只能降级使用 Vue 1.x，但官方早已停止维护。实际项目中，大多数团队直接放弃 IE8，即使是国内银行类系统，也普遍以 IE9 为最低门槛。

代码层面，Vue 2 源码里没有任何针对 IE8 的降级逻辑，构建产物也不包含 polyfill 来补全缺失特性。所以哪怕你引入了 es5-shim 也没用，核心的属性拦截机制依然搞不定。



现在主流做法是通过构建工具配置目标浏览器范围，比如在 `.browserslistrc` 中声明 `> 1%, not dead, ie >= 9`，让 Babel 自动注入所需 polyfill，同时确保打包结果兼容到 IE9。

在 Vue 组件中写 name 选项有什么作用？

组件的 `name` 选项不只是为了标识，它在几个关键场景里起着实际作用。

递归组件调用时，必须通过 `name` 来实现自引用。比如一个树形结构组件 `TreeNode`，它要在模板里调用自己，就得靠 `name: 'TreeNode'` 才能识别。

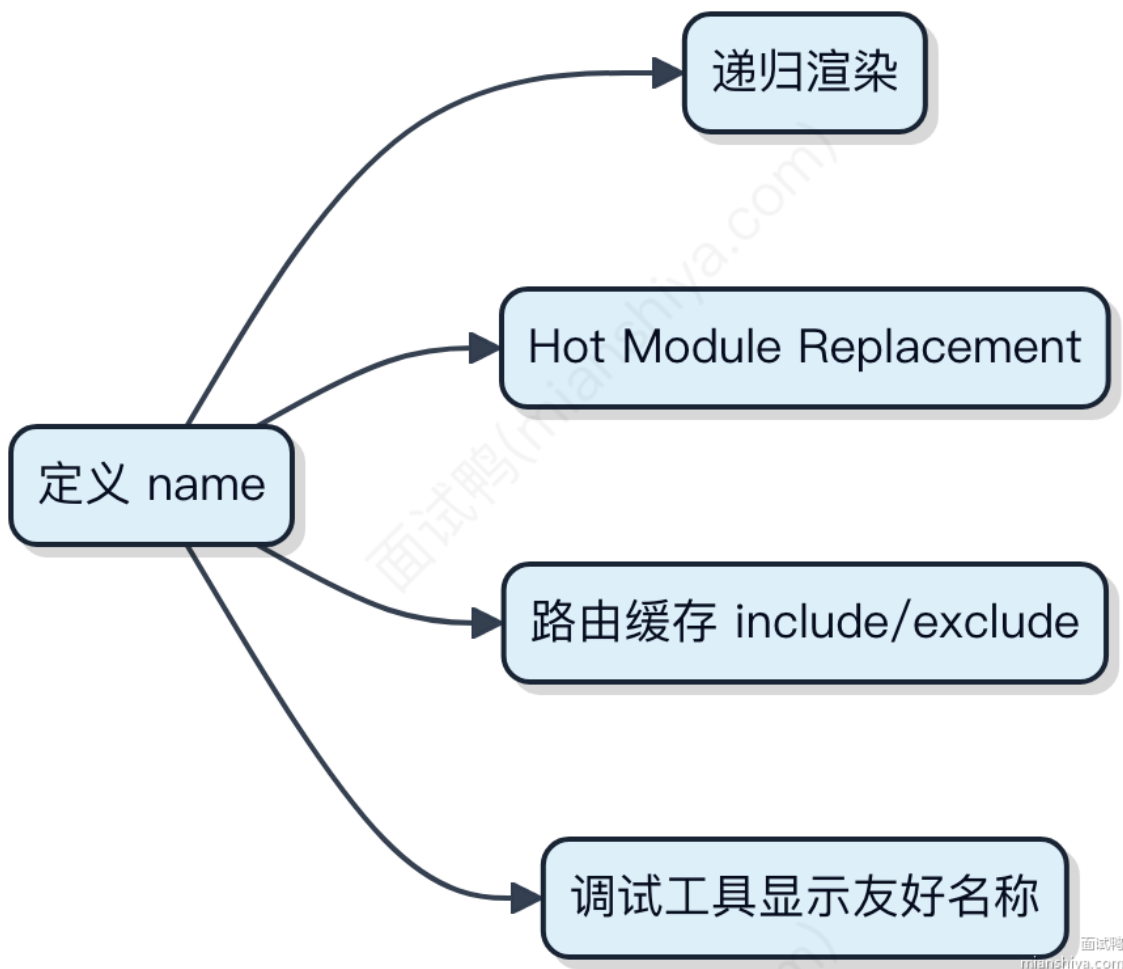
```
<template>
  <div>
    <TreeNode v-for="child in children" :key="child.id" />
  </div>
</template>
<script>
export default {
  name: 'TreeNode',
  props: ['children']
}
</script>
```

热重载也依赖 `name`。开发环境下修改组件代码，Vue 通过这个名字判断是否是同一个组件，保持状态不丢失。没有 `name`，每次保存都会重新渲染，体验很差。

路由跳转时也有用。Vue Router 支持基于组件名称的导航守卫匹配，比如 `beforeRouteLeave` 或 `router-view` 的 `include/exclude` 缓存控制，都是拿 `name` 当标识。

单文件组件虽然可以通过文件名推断 `name`，但显式声明更可靠。特别是经过构建工具处理后，文件名可能被压缩或改变，直接定义最保险。

- 1) 递归组件需要自引用
- 2) HMR 热更新靠它维持状态
- 3) 路由和缓存机制依赖名称匹配



Vue 的 v-show 和 v-if 有什么区别？使用场景分别是什么？

v-show 和 v-if 都是用来控制元素显示隐藏的指令，但实现方式完全不同。

v-if 是“真正”的条件渲染，它会根据表达式真假决定是否创建或销毁 DOM 节点。比如 `v-if="visible"` 为 false 时，元素压根不经过渲染，DOM 树里根本不存在这个节点，切换时要重新执行组件的挂载或销毁流程。

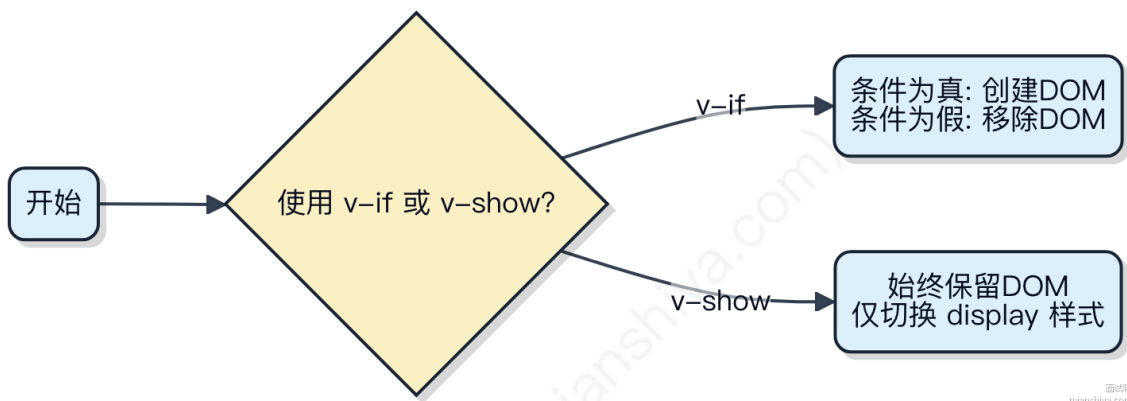
v-show 则简单粗暴得多，不管条件如何，元素始终会被渲染到 DOM 中，只是通过 CSS 的 `display: none` 来控制视觉上的显隐。所以它的初始渲染开销更大一点，但切换成本极低，没有 DOM 重建过程。

1) v-if 适合场景：切换不频繁、初始化性能要求高。比如用户权限不同展示完全不同的模块，用 v-if 可以避免无用的 DOM 占用内存。

2) v-show 适合场景：需要频繁切换显示状态。比如一个常驻的弹窗或标签页，反复开关时用 v-show 能避免重复渲染，性能更好。

注意，v-show 不支持 `<template>` 标签，也不能和 v-else 搭配使用，这是语法限制。

如果用在组件上，v-if 会触发整个子组件的卸载与重建，而 v-show 只是传个 prop 控制内部 display，**渲染机制**的不同决定了它们的适用边界。



怎么使 CSS 样式只在当前 Vue 组件中生效?

在 Vue 组件里让 CSS 样式只作用于当前组件，靠的是**作用域样式**。你只需要在 `<style>` 标签上加上 `scoped` 属性，Vue 就会自动处理样式隔离。

它背后的原理其实挺简单。加了 `scoped` 之后，Vue 编译模板时会给当前组件的每个元素都添加一个唯一的属性，比如 `data-v-f3f4394e`，然后把你的样式选择器也改写成带这个属性的，确保样式只匹配当前组件内的元素。

举个例子：

```
<template>
  <div class="example">Hello</div>
</template>

<style scoped>
.example {
  color: red;
}
</style>
```

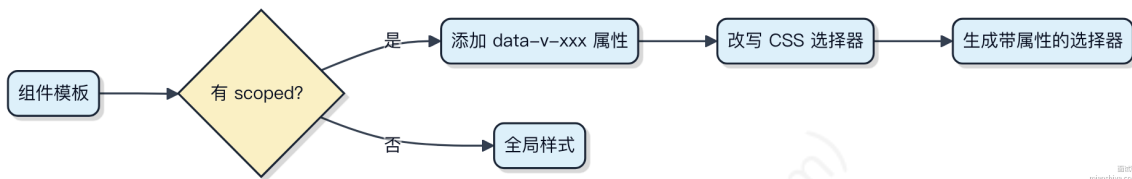
会被编译成类似这样：

```
.example[data-v-f3f4394e] {
  color: red;
}
```

这种方式基本能搞定大多数场景，而且不需要你手动管理类名冲突。但要注意一点，如果你用了深度选择器，比如想影响到子组件的内部元素，就得用 `:deep()` 包一层。例如：

```
:deep(.el-button) {
  margin: 10px;
}
```

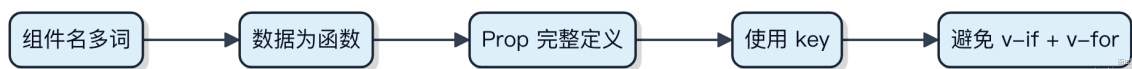
另外，如果组件有 `v-html` 或者动态插入的内容，这些内容不会自动带上作用域属性，样式可能就压根不经过 `scoped` 控制，得自己额外处理。



是否了解 Vue 官方的风格指南？请列举其中的一些规则

Vue 官方风格指南本质上是团队协作的编码规范，目标是统一代码风格、减少错误、提升可维护性。它把规则分成 A、B、C、D 四个等级，A 是必要，D 是谨慎使用。

- 1) 优先级 A 的规则必须遵守。比如组件名应该始终是多单词，避免跟原生 HTML 标签冲突，像 `Button` 就不行，得写成 `BaseButton` 或 `MyButton`。数据必须是函数，否则组件复用时会共享同一个对象引用，造成状态污染。
- 2) 优先级 B 的规则是强烈推荐。比如指令缩写要一致，要么都用 `:` 要么都不用；Prop 定义应该完整，明确类型、是否必传、默认值，不能只写一个字符串。还有就是要有 `key`，特别是在 `v-for` 中，用 `index` 当 `key` 在列表动态变化时容易导致 DOM 更新错乱。
- 3) 优先级 C 是推荐一致性。比如文件名统一用 `PascalCase` 或 `kebab-case`，整个项目要保持一致。组件顺序在 `.vue` 文件中建议按 `<template>`、`<script>`、`<style>` 排列，方便快速定位。
- 4) 优先级 D 是慎用。比如 `v-if` 和 `v-for` 不要同时用在同一个元素上，因为 `v-for` 优先级更高，会导致每次循环都执行判断，性能很差，应该通过计算属性提前过滤。



这些规则看着琐碎，但在多人协作和项目长期维护中能省不少沟通成本。

Vue 中 `created` 和 `mounted` 生命周期钩子有什么区别？

Vue 实例的生命周期里，`created` 和 `mounted` 是两个关键阶段，它们的触发时机和适用场景完全不同。

- 1) `created` 阶段，实例完成数据观测、属性和方法的运算，**响应式系统**已经建立，但此时 `$el` 还没生成，页面上的 DOM 节点压根不存在。你可以在这里发起数据请求、初始化非 DOM 依赖的状态，比如调用接口拉取列表数据，或者设置定时器。
- 2) `mounted` 阶段，Vue 已经把编译好的模板挂载到页面上，`$el` 正式指向真实 DOM 元素。这时候才能安全地访问和操作 DOM，比如用 `document.querySelector` 拿元素，或者初始化依赖 DOM 的第三方库（像 `ECharts`、`Swiper` 这类）。服务端渲染时，`mounted` 在客户端才会执行，这一点要注意。

举个例子：你在一个组件里要根据容器宽度初始化图表，这事只能在 `mounted` 做，`created` 里查不到 DOM 宽度。反过来，如果只是从后端拉配置项，那在 `created` 发请求更早，能缩短等待时间。



简单说，数据逻辑放 `created`，DOM 交互放 `mounted`。搞混了就可能拿到 `null` 的元素，或者在不该发请求的地方重复调用。

为什么 Vue 要求组件模板只能有一个根元素？

Vue 在编译模板时需要生成一个 **虚拟 DOM 树**，这个树结构必须有且仅有一个根节点，才能被正确挂载和更新。如果模板里有多个同级元素，Vue 不知道该以谁为根，整个渲染流程就乱了。

1) 在单文件组件 (.vue) 中，template 下必须只有一个根标签。比如你写：

```
<template>
  <div class="container">
    <h1>标题</h1>
    <p>内容</p>
  </div>
</template>
```

这是合法的。但如果你把 `div.container` 去掉，直接两个同级标签，Vue 编译就会报错。

2) 这个限制从 Vue 2 一直存在，到 Vue 3 才通过 **Fragment** 特性部分解除了。Vue 3 支持模板写多个根节点，前提是使用运行时编译或构建工具配合，但 `<template>` 单独使用时依然推荐单根，避免边界问题。

3) 实际开发中，像 Element Plus、Ant Design Vue 这些 UI 框架的组件内部也都遵循单根惯例，方便父组件统一控制样式和布局。

如果非要用多根，可以用 Vue 3 的 `<teleport>` 或者逻辑拆分，但日常开发老老实实包一层 `div` 最稳妥。毕竟工具链、调试工具、CSS 作用域都按单根假设来设计的。



Vue 父子组件之间传值有哪些方式？

父子组件传值最常用的还是通过 **props** 和 **事件** 这一套组合来搞。

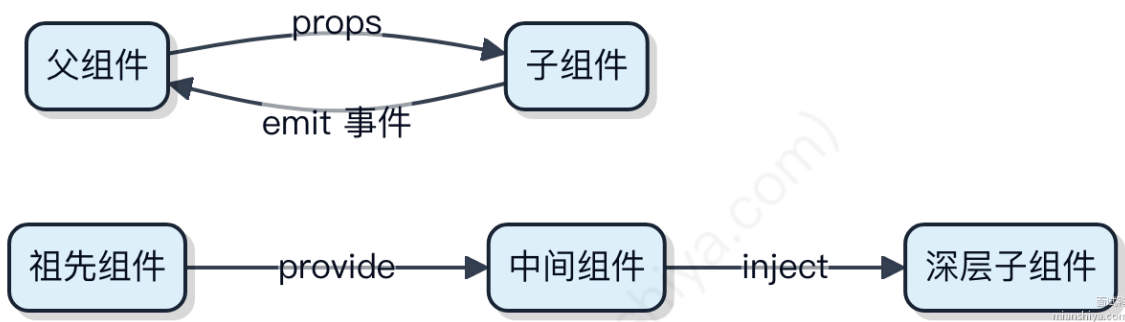
父组件往子组件传数据，直接用 props 定义接收字段就行。比如父组件模板里写 `<user-card :name="userName" :age="userAge" />`，子组件声明 `props: ['name', 'age']` 就能拿到。

子组件想通知父组件，就用 `$emit` 触发事件。父组件在模板里监听，比如 `<user-card @update-name="handleNameChange" />`，子组件内部调 `this.$emit('update-name', 'newName')`，父组件的 `handleNameChange` 方法就会执行。

如果嵌套深了，一层层传太麻烦，可以考虑 `provide / inject`。祖先组件用 `provide` 提供数据，后代组件用 `inject` 直接获取，跨多层都行。但注意这不是响应式的，除非你传的是个响应式对象。

还有种情况是父组件要直接调子组件的方法或访问属性，可以用 `ref`。父组件给子组件标记

`ref="childComp"`，然后通过 `this.$refs.childComp` 调用它暴露出来的方法。不过这种紧耦合用多了维护会难受，一般用于表单提交、弹窗控制这类明确的父子操作。



简单场景优先 props + emit，深度嵌套考虑 provide/inject，特殊操作再用 ref。别一上来就上 Vuex 或 Pinia，小题大做没必要。

有哪些定义 Vue 组件模板的方法？

写 Vue 组件模板，其实就那么几种主流方式，选哪种主要看项目规模和团队习惯。

1) 最直接的就是在单文件组件 .vue 里用 `<template>` 标签写。这是最常见的做法，结构清晰，配合 VS Code 的 Volar 插件，有语法高亮和自动补全，开发体验好。

```
<template>
  <div class="header">Hello {{ title }}</div>
</template>
```

2) 字符串模板，就是把模板写成一个 JavaScript 字符串，比如在 `render` 函数里用，或者通过 `template` 选项传进去。这种方式灵活性高，适合动态生成模板的场景，但可读性差，调试麻烦，一般不推荐日常使用。

3) `render` 函数，用 JavaScript 写模板逻辑，可以完全发挥 JS 的编程能力。比如你要根据数据动态生成大量相似结构，用 `render` 函数会更灵活。Vue 3 里结合 JSX 语法，写起来更顺。

```
render() {
  return h('div', this.title)
}
```

4) DOM 模板，直接从页面 HTML 中某个元素的 `innerHTML` 读取模板，比如用 `<script type="text/x-template">` 包裹。这种方式现在很少见了，多用于老项目迁移，或者 CMS 这类需要从服务端注入模板的场景。

总的来说，**单文件组件**是现代 Vue 开发的标准做法，90% 以上的项目都这么干。其他方式属于特定场景的补充，知道就行，真遇到再说。

为什么 Vue 写组件时可以放在 .vue 文件里？可以使用其他文件后缀吗？

.vue 文件其实是个语法糖容器，把模板、脚本、样式打包在一起，方便组件化开发。构建工具比如 Vite 或 Webpack 配合 Vue Loader 能解析它，拆出 `template`、`script`、`style` 三块分别处理。`template` 会编译成 `render` 函数，`script` 是组件逻辑，`style` 处理 `scoped` 等特性。

你当然可以不用 .vue 后缀。Vue 支持用普通 .js 或 .ts 文件通过 `defineComponent` 创建组件，适合简单场景或函数式组件。也可以用 .jsx/.tsx 写，走 `render` 函数路线，灵活性更高，但模板提示和 `scoped` 样式就得自己搞。

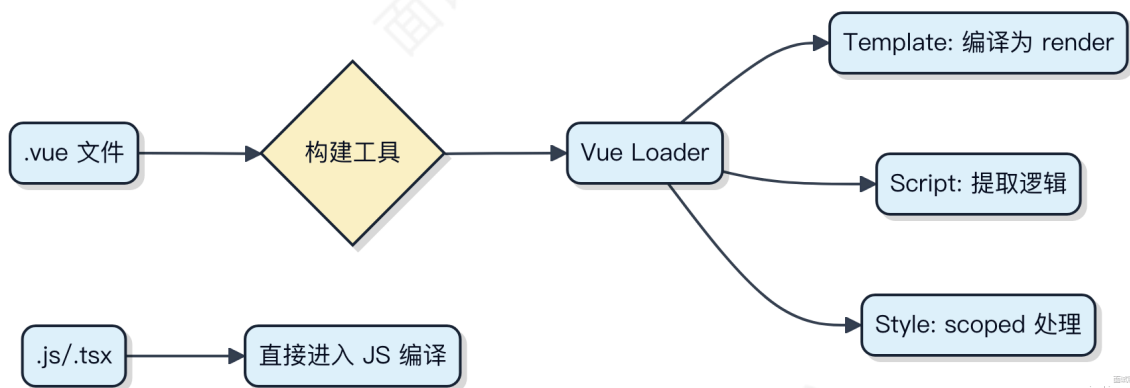
关键在于构建配置能不能识别和处理对应文件。比如你想用 .vue.ts 当后缀，那 loader 规则得配好，否则打包阶段就报错。实际项目里没人这么干，没必要增加复杂度。

代码示例：

```
import { defineComponent } from 'vue'

export default defineComponent({
  name: 'HelloWorld',
  setup() {
    return () => <div>Hello</div>
  }
})
```

这种写法压根不经过 .vue 解析流程，直接走 JS 编译通道。



Vue 中组件、插件、插槽三个概念的区别是什么？

组件是 Vue 里复用 UI 和逻辑的基本单元，你可以把它看作一个自定义的 HTML 标签。比如你写个 `<UserCard>`，里面封装了用户的头像、昵称展示和一些交互逻辑，anywhere 需要用到就直接写这个标签就行。

插件解决的是全局能力扩展的问题。比如你用了 `vue-router` 或 `Pinia`，它们不是某个具体页面元素，而是要注入路由能力或者状态管理能力到整个应用。插件一般通过 `app.use()` 安装，可以注册全局组件、指令，也可以往 app 实例上挂方法。

插槽 (slot) 是组件内部预留的内容分发位置，用来增强组件的灵活性。父组件可以往子组件的 slot 里塞内容，子组件决定结构，内容由父级定。比如一个 `<Modal>` 组件，header 和 footer 固定，中间这块动态内容就可以用 slot 让外部传。

1) 默认插槽：只一个 slot 标签，不加 name 2) 具名插槽：用 name 属性区分多个插入点 3) 作用域插槽：子组件把数据传给父级 slot 内容使用

组件负责“长什么样、怎么动”，插件负责“加能力”，插槽负责“哪里能被定制”。三者分工明确，搞清楚各自职责边界，才能写出高内聚、可复用的 Vue 应用。

什么是 Vue 的 v-model? 有什么作用？

v-model 是 Vue 里处理表单元素双向绑定的语法糖，不用它的话，你得手动监听 input 事件再更新数据，特别麻烦。

它的本质是 `:value` 和 `@input` 的简写组合。比如在 `<input>` 上用 `v-model`, Vue 会根据元素类型自动选择正确的事件和属性来同步数据。输入框用 `value` 和 `input` 事件, checkbox 用 `checked` 和 `change` 事件。

1) 对于原生表单元素, 直接绑定数据

```
<input v-model="username" />
<!-- 等价于 -->
<input :value="username" @input="username = $event.target.value" />
```

2) 在组件上使用时, `v-model` 默认会传递一个 `modelValue` 的 prop, 并监听 `update:modelValue` 事件。你可以通过参数自定义这个名称, 比如 `v-model:title` 就会用 `title` 和 `update:title`。

3) 自定义组件内部需要配合 `emits: ['update:modelValue']` 声明, 并在合适时机触发更新。

```
<!-- Child.vue -->
<script setup>
defineProps(['modelValue'])
defineEmits(['update:modelValue'])
</script>
<template>
  <input
    :value="modelValue"
    @input="$emit('update:modelValue', $event.target.value)"
  />
</template>
```

现在主流都是用 Composition API 配合 `v-model`, 在 `<script setup>` 里处理逻辑更清晰。尤其结合 `computed` 的 `get/set`, 能实现更复杂的双向同步逻辑。

[Vue 的模板语法使用的是哪个 Web 模板引擎? 介绍下该模板引擎](#)

Vue 的模板语法不是基于任何第三方 Web 模板引擎, 它是自研的一套声明式模板系统, 直接构建在浏览器的 DOM 解析能力之上。

这套模板本质上是 HTML 的扩展, 通过指令 (如 `v-if`、`v-for`、`v-bind`) 来建立数据与视图的绑定关系。浏览器能原生解析 HTML, 所以 Vue 模板可以直接被当作 DOM 处理, 不需要预编译 (开发阶段可选, 但运行时可用字符串模板)。

1) 模板在运行时会被编译成 **渲染函数** (render function), 这个过程由 Vue 的编译器完成。比如 `<div v-if="show">hello</div>` 最终会转成类似 `show ? createElement('div', 'hello') : null` 的 JS 代码。

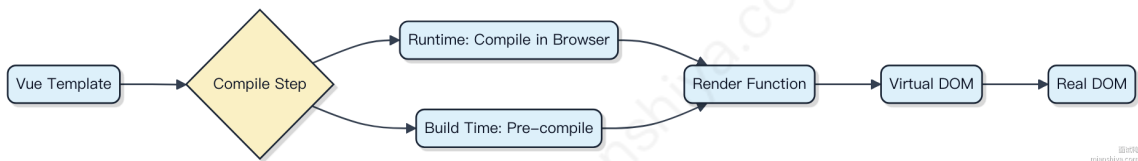
2) Vue 提供两种使用方式: 直接在页面写 `<template>` 字符串, 或者用 `*.vue` 单文件组件配合构建工具预编译。后者在构建时就把模板转成了渲染函数, 运行时不再需要编译器, 包更小。

3) 它不依赖 Mustache、Handlebars 或 Twig 这类传统模板引擎。那些是字符串替换模型, 而 Vue 是基于响应式系统 + 虚拟 DOM 的动态更新机制, 更新粒度更细, 性能更好。

代码示例如下:

```
// 模板
const template = `<p v-if="visible">{{ message }}</p>`
```

```
// 编译后生成的渲染函数（简化）
render(h) {
  return this.visible ? h('p', this.message) : null
}
```



Vue 项目中如何监听键盘事件？

监听键盘事件在 Vue 里其实挺直接，主要靠 `v-on` 指令，也就是 `@` 语法糖。你可以绑定原生的 `keydown`、`keyup` 这些事件，直接在模板里写。

比如想监听回车键提交，可以这样：

```
<input @keyup.enter="submitForm" />
```

这里 `.enter` 是按键修饰符，Vue 内置了常用键的别名，像 `.tab`、`.esc`、`.space`、`.ctrl` 等等。按 Enter 的时候才会触发 `submitForm` 方法。

如果要监听组合键，比如 `Ctrl+S`，可以用：

```
<div @keyup.ctrl.s="saveData">按 Ctrl+S 保存</div>
```

注意修饰符顺序不能错，`.ctrl.s` 才对，`.s.ctrl` 就不行。

自定义键码也行，但不推荐。Vue 2 支持 `.13` 这种数字写法，Vue 3 已经废弃了，建议用语义化的修饰符。

还有一个细节，有些场景下你可能想让事件只触发一次，比如调试快捷键，可以配合 `.once` 修饰符：

```
<div @keydown.esc.once="closeModal">按一次 Esc 关闭</div>
```

全局监听的话，比如监听整个 document 的按键，就不能用模板语法了，得在 `mounted` 里手动绑定，记得在 `beforeUnmount` 解绑，避免内存泄漏。

```
mounted() {
  document.addEventListener('keydown', this.handleKeydown)
},
beforeUnmount() {
  document.removeEventListener('keydown', this.handleKeydown)
}
```

这种全局监听适合做快捷键系统，比如你在 **Element Plus** 或 **Ant Design Vue** 里看到的全局快捷操作，基本都是这么搞的。

在 Vue 组件中如何获取当前的路由信息？

用 `useRoute` 和 `useRouter` 这两组合式 API 最直接。你在 `setup` 里想拿路径、参数、查询字符串，直接调 `useRoute()` 就行。

- 1) `useRoute` 返回的是当前激活的路由状态，比如 `route.path`、`route.params`、`route.query` 都能直接读。页面跳转后它会自动更新，响应式的，模板里用也不用额外处理。
- 2) `useRouter` 拿的是路由实例本身，你要做跳转、前进后退，就用它提供的 `push`、`replace` 方法。比如 `router.push('/home')` 或带参数 `router.push({ name: 'User', params: { id: 886 } })`。

代码长这样：

```
import { useRoute, useRouter } from 'vue-router'

export default {
  setup() {
    const route = useRoute()
    const router = useRouter()

    console.log(route.path) // 直接访问路径
    console.log(route.query.id) // 拿 URL 查询参数

    const goHome = () => router.push('/')

    return { goHome }
  }
}
```

注意别在模板外面直接用 `route` 变量，得确保是在 `setup` 或 `<script setup>` 里调用。这两 hook 压根不经过 Vue 的选项式逻辑，只在组合式 API 环境生效。

老项目如果还在用选项式 API，可以用 `this.$route` 和 `this.$router`，但新代码建议全走 `useRoute` / `useRouter`。

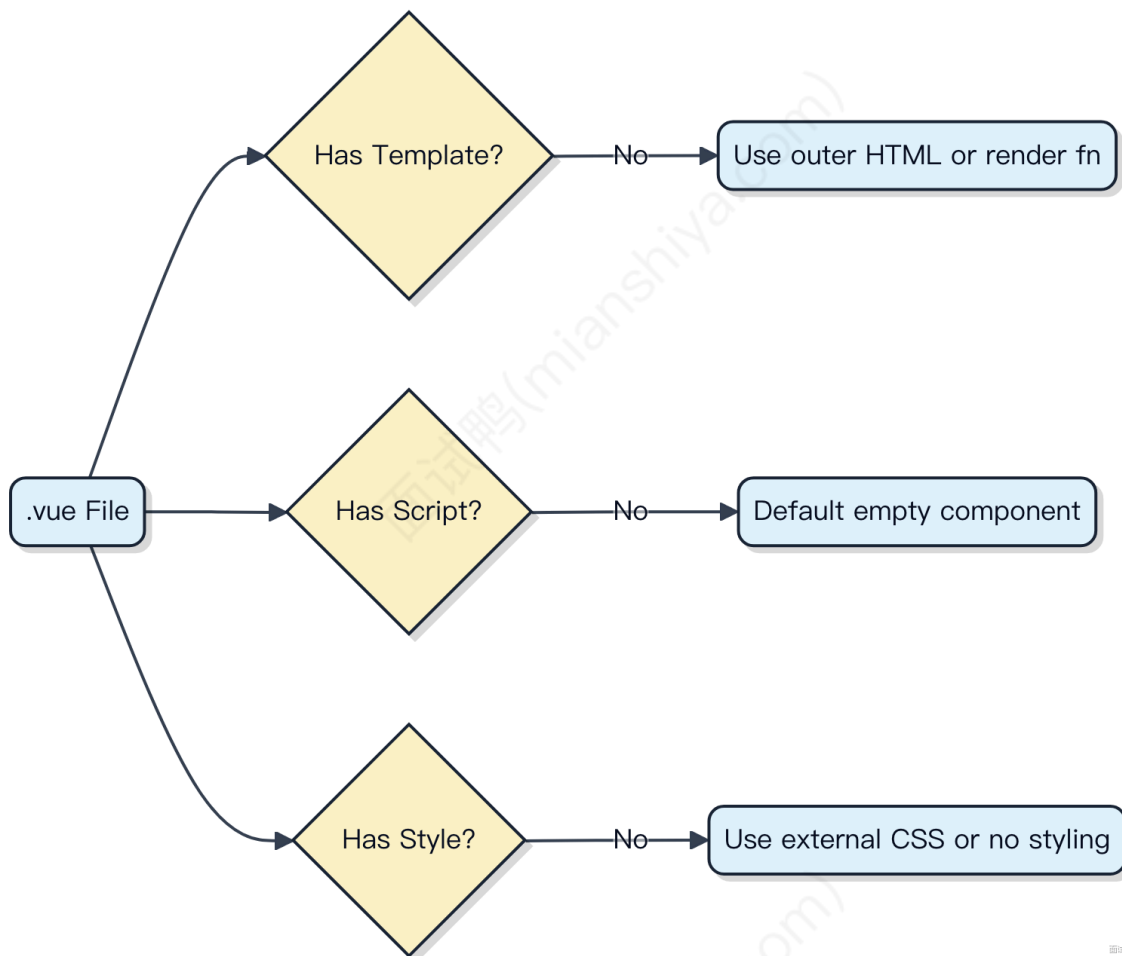
在 .vue 文件中，style 和 script 必须要写么？为什么？

一个 `.vue` 文件其实就是一个 Vue 单文件组件（SFC），它由三部分组成：`template`、`script` 和 `style`。但这三部分都不是强制要求必须写的。

- 1) `template` 不写的话，Vue 会尝试从挂载的 DOM 元素里找模板，或者你可以在 `script` 里通过 `render` 函数定义渲染逻辑。比如用 JSX 或直接 `createElement`，这时候根本不需要写 `template` 标签。
- 2) `script` 部分看起来像是必须的，但其实你可以用 `<script setup>` 甚至完全省略 `script` 标签，只要组件逻辑足够简单。比如你只写个静态页面，没有响应式数据、没有方法，那 `script` 完全可以不写。Vue 会默认导出一个空的组件配置。
- 3) `style` 更不是必须的。你不加 `style` 标签，组件就没有 `scoped` 或全局样式。但很多场景确实不需要样式，比如布局组件或纯逻辑组件。就算要样式，也可以通过外部 CSS 文件引入，不一定非得写在 `.vue` 里面。

所以这三块都是可选的，Vue 只关心最终能拿到组件的渲染函数和配置。实际项目里，像 `Nuxt.js` 或一些 UI 库里的组件，经常能看到只有 `template` 或只有 `script` 的情况。

要不要写，取决于你当前组件有没有对应职责。没逻辑就不写 script，没样式就不写 style，很自然。



这种设计让 SFC 更灵活，而不是被结构绑架。

什么是 Vue 的 nextTick? 有什么作用?

Vue 的 nextTick 利用的是浏览器的异步任务机制，把回调函数推迟到 DOM 更新周期结束后再执行。你每次修改响应式数据，Vue 不会立刻更新 DOM，而是把这些变更缓存起来，等到当前事件循环末尾统一处理。这时候如果你需要在 DOM 更新后做些操作，比如获取更新后的元素尺寸，就得靠 nextTick。

它背后的核心是微任务优先策略。Vue 会优先尝试用 Promise.then、MutationObserver 或 setImmediate，降级到 setTimeout。所以你调 nextTick，其实是把回调塞进了微任务队列，确保它在所有同步的 DOM 变更完成后、下一个宏任务开始前执行。

常见使用场景像：手动触发 \$refs 计算布局，或者在动态插入列表后获取滚动高度。不用它的话，直接读 DOM 很可能拿到的是旧的、还没刷新的状态。

代码上就这么用：

```
this.$nextTick(() => {  
  // 这里可以安全访问更新后的 DOM  
})
```

```
console.log(this.$refs.container.scrollHeight)
})
```

或者用 async/await:

```
await this.$nextTick()
console.log('DOM 已更新')
```

- 1) 数据变更后需操作真实 DOM 的场景必须用
- 2) 它不解决跨组件通信，也不是定时器替代品
- 3) 理解它的本质是微任务调度，不是 Vue 自己实现的 DOM 监听



Vue 中 MVVM、MVC 和 MVP 模式的区别是什么？

MVVM、MVC 和 MVP 都是 UI 架构模式，解决的是视图和数据之间的耦合问题。它们的差异主要体现在职责划分和数据流控制上。

MVC 里，View 直接和 Model 通信，也和 Controller 通信。用户操作发给 Controller，Controller 处理完更新 Model，Model 变化后主动通知 View 刷新。这种模式在 Web 前端容易导致 View 和 Model 耦合过重，典型如早期 jQuery 项目里到处是 \$.ajax 更新 DOM。

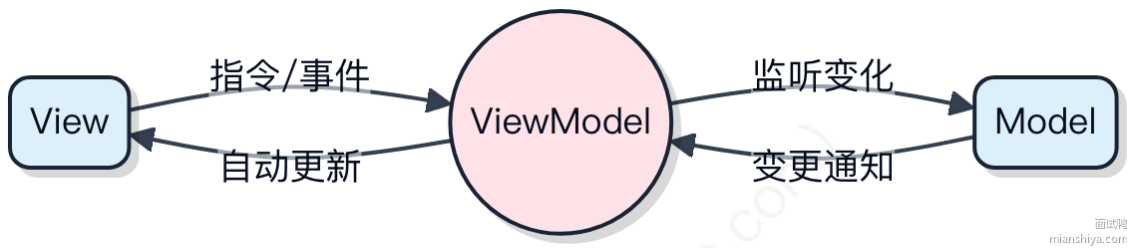
MVP 改进了这一点，引入 Presenter 作为中间人。View 不直接拿 Model，所有逻辑都通过 Presenter 转发。Presenter 持有 View 接口和 Model，处理业务后回调 View 更新。它解决了部分耦合，但样板代码多，Android 开发早期常用这种模式。

Vue 用的是 MVVM。**ViewModel** 是关键角色，它通过双向绑定连接 View 和 Model。你改数据，视图自动变；视图交互变了，数据也自动更新。开发者不用手动操作 DOM，Vue 内部用响应式系统追踪依赖，数据变时自动派发更新。

```
new Vue({
  data: { message: 'Hello' },
  template: `<div @click="message = 'Hi'">{{ message }}</div>`
})
```

这个例子中，data 是 Model，template 是 View，Vue 实例就是 ViewModel。点击事件修改 message，DOM 自动响应，整个过程压根不经过手动操作。

- 1) MVC：三者耦合较高，适合简单场景
- 2) MVP：Presenter 解耦，测试友好但代码量大
- 3) MVVM：靠响应式和数据绑定减少模板代码，Vue、Angular 都走这条路



怎么在组件初始化时立即调用 Vue 的 watch 函数？

Vue 的 watch 默认是懒执行的，监听的数据变了才会触发。想在组件初始化时就跑一遍逻辑，得手动处理。

1) 用 `immediate: true` 选项，这是最直接的办法。watch 创建后立刻执行一次回调，不用等数据变化。

```
watch: {
  someData: {
    handler(val) {
      console.log('值是', val)
    },
    immediate: true
  }
}
```

2) 如果用的是 Vue 3 的 watch API，写法类似，传个配置对象带上 `immediate`。

```
watch(source, (val) => {
  // 回调逻辑
}, { immediate: true })
```

3) 另一个思路是把这部分逻辑抽成函数，setup 或 created 阶段先调一次，再放进 watch 里复用。适合逻辑复杂、不想混在 watch 配置里的场景。

```
function update() { /* 处理逻辑 */ }

// 组件启动先执行
update()

watch(someRef, update)
```

注意别滥用 `immediate`，有些场景其实用 `computed` 更合适。watch 带 `immediate` 是为了响应式地同步副作用，`computed` 才是派生状态的首选。如果只是算个值，别拿 watch 搞脏活累活。

Vue 2.0 的 v-html 指令不再支持使用过滤器，还有哪些处理 HTML 内容的方法？

Vue 2.0 的 `v-html` 指令确实不再支持过滤器，这是设计上的明确取舍。过滤器在 Vue 3 中已被彻底移除，所以得用更现代的方式处理动态 HTML 内容。

最直接的替代方案是在 **JavaScript 中预处理数据**，把需要展示的 HTML 字符串提前拼好。比如从接口拿到富文本内容后，用 DOMParser 或正则做清洗，再绑定到模板。这种方式逻辑清晰，也方便测试。

如果涉及用户输入的 HTML，**必须做 XSS 防护**，不能直接 v-html 输出。推荐用 **DOMPurify** 这类库做净化，它能有效过滤 script 标签和危险属性。像 Element UI 的富文本组件内部就用了类似机制。

也可以封装成计算属性或方法，在模板里调用：

```
computed: {
  safeHTML() {
    return DOMPurify.sanitize(this.rawHTML)
  }
}
```

然后模板中使用 `v-html="safeHTML"`。这样既保持了安全性，又实现了动态渲染。

还有一种思路是**用组件代替原始 HTML 拼接**。比如把富文本拆成多个子组件，通过 props 传递结构化数据，由组件决定如何渲染。这种方式更重，但安全性和可维护性更好，适合内容结构固定的场景。

总之，v-html + 过滤器的写法已经过时，现在主流是“**数据预处理 + 净化库 + 结构化渲染**”这套组合拳。

Vue 实例在挂载过程中发生了什么？

Vue 实例从创建到挂载，核心是把数据驱动的视图真正渲染到页面上。整个过程在调用 `$mount` 时启动，如果实例没指定 `el`，得手动调用它。

- 1) 先编译模板。如果没有提供 render 函数，Vue 会把 template 或 innerHTML 转成 render 函数。这一步在开发环境常见，生产环境建议直接用 render 或单文件组件避免运行时编译。
- 2) 执行 mountComponent，这里会创建一个 **Watcher** 实例。这个 Watcher 会立即触发一次 getter，从而触发所有响应式数据的依赖收集。同时，它会调用 `_render` 生成 VNode，再通过 `_update` 把 VNode 转成真实 DOM 插入页面。
- 3) 首次更新时，`_update` 方法会走 diff 算法对比新旧 VNode。但第一次没有旧节点，所以直接根据 VNode 的类型递归创建真实元素。如果是普通标签就 `createElement`，是组件则触发组件实例化和挂载。



整个流程里，数据变化不会立刻更新 DOM，而是异步批量处理。这也是为什么你连续改 10 次 count，DOM 只更新一次。只要搞懂“**响应式数据 + Watcher + Virtual DOM**”这条链路，挂载机制就通了。

什么是 Vue 的单向数据流和双向数据流？

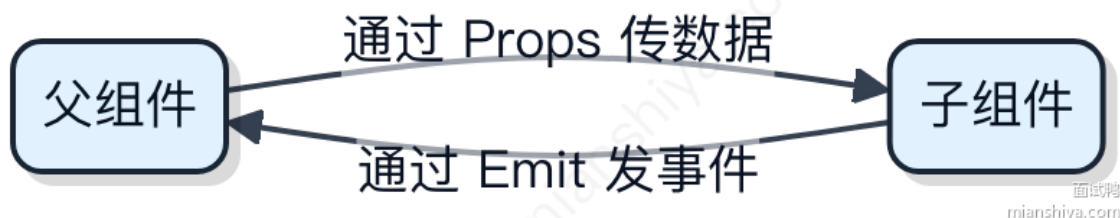
Vue 的单向数据流指的是数据从父组件流向子组件，子组件不能直接修改 props。你只能通过事件通知父组件来更新数据，这样能保证状态变化可追踪。

比如父组件传 `value` 给子组件，子组件想改这个值，得 emit 一个 `update:value` 事件，父组件监听后自己改。这种模式让数据流向清晰，调试更容易。

双向数据流看起来是父子之间“同步”修改，其实是语法糖。最常见的就是 `v-model`。你在组件上写 `v-model="msg"`，等价于 `:modelValue="msg" @update:modelValue`。

它底层还是靠事件驱动更新，不是真的双向绑定内存引用。所以本质上 Vue 始终维持着单向数据流的规则，**v-model** 只是简化写法。

1) 父组件传递数据给子组件 (props) 2) 子组件触发事件通知变更 3) 父组件响应事件并更新数据 4) 新数据再次下传，形成闭环



自定义组件实现 v-model 也很简单：

```
<!-- 子组件 -->
<input
  :value="modelValue"
  @input="$emit('update:modelValue', $event.target.value)"
>
```

不建议绕过单向流用 ref 直接改，容易导致状态混乱。尤其在多人协作或大型项目里，守住单向流能让系统更可控。

什么是 Vue 的自定义指令？自定义指令的应用场景有哪些？

Vue 的自定义指令让你能直接操作 DOM，适合处理那些需要底层 DOM 控制的逻辑。它不像组件那样构建 UI 结构，而是专注于封装**重复性的 DOM 操作行为**。

1) 你可以在元素上绑定一个 **v-** 开头的指令，比如 **v-focus** 让输入框自动聚焦，或者 **v-scroll** 监听滚动事件。这些逻辑如果写在每个组件的生命周期里会很重复，抽成指令就干净多了。

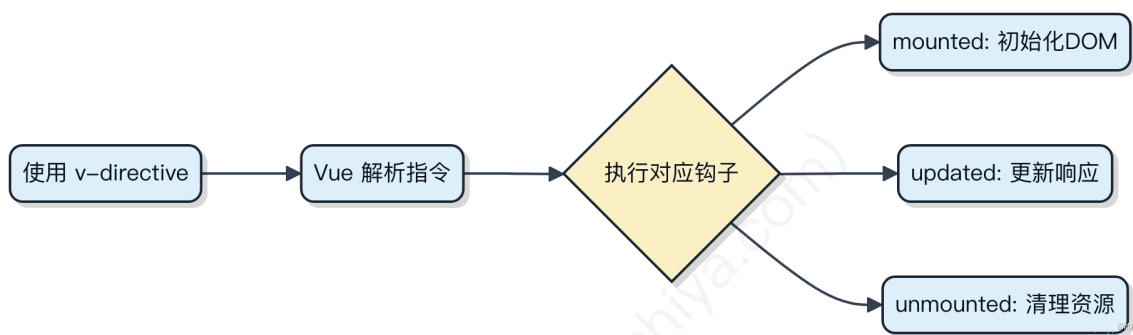
2) 常见的应用场景包括权限控制，比如 **v-permission="admin"** 控制按钮是否显示；还有防抖节流的 **v-debounce**，避免按钮被频繁点击；再比如图片懒加载 **v-lazy**，页面滚动时才加载可视区域的图片，提升性能。

3) 指令有五个钩子函数：**created**、**beforeMount**、**mounted**、**updated**、**unmounted**。最常用的是 **mounted** 做初始化 DOM 操作，**unmounted** 清理事件监听。比如实现一个拖拽指令 **v-draggable**，就在 **mounted** 里绑定 **mousedown** 和 **mousemove** 事件。

代码示例：

```
app.directive('focus', {
  mounted(el) {
    el.focus()
  }
})
```

这类需求在中后台系统特别常见，像表单自动聚焦、权限按钮、长列表优化，用自定义指令能把逻辑拆得更清晰。本质上它是 Vue 提供了一种**关注点分离**手段，把 DOM 操作从业务逻辑里剥离出来。



本资源来自面试鸭: <https://www.mianshiya.com>

推荐更多免费学编程资源:

1. 编程导航学习网站: [学编程、做项目、拿 Offer!](#)
2. 企业高频面试题库: [开始刷题, 面试遇原题!](#)
3. 精选简历模板大全: [1 分钟搞定简历!](#)
4. AI 资源导航网站: [获取最新 AI 黑科技!](#)
5. 1 对 1 模拟面试: [随时随地提升面试能力](#)